

Extension of a RISC-V Core with Custom Matrix Multiplication instructions

Erklärung zur Abschlussarbeit

Hiermit versichere ich, dass die eingereichte Ausarbeitung von mir persönlich verfasst und meine eigene individuelle Prüfungsleistung ist.

Ich versichere, dass die Ausarbeitung und auch keine Teile davon durch künstliche Intelligenz (KI) dergestalt erstellt wurden, dass das KI-Werk bzw. KI-Werkteile meine eigene Prüfungsleistung ersetzen. Ich versichere, KI allenfalls eingesetzt zu haben, um einen von KI für meine Aufgabenstellung ausgearbeiteten Lösungsvorschlag kritisch zu beurteilen und/oder einen Überblick über Aspekte zu erhalten, die für die von mir in Eigenleistung zu erbringende Prüfungsleistung relevant sein könnten. Soweit durch die Aufgabenstellung bzw. Hinweise der Prüfenden der Einsatz von KI vorgegeben ist, sind die von KI erzeugten Werkteile von mir in der Arbeit entsprechend gekennzeichnet. Mir ist bewusst, dass die von KI erzeugten Werkteile auf ihre Validität zu überprüfen und nicht zitierfähig sind.

Ebenso versichere ich, dass diese Arbeit oder Teile daraus weder von mir selbst noch von anderen als Leistungsnachweise andernorts eingereicht wurden.

Wörtliche oder sinngemäße Übernahmen aus anderen Schriften und Veröffentlichungen in gedruckter oder elektronischer Form sind gekennzeichnet. Sämtliche Literatur und sonstige Quellen sind nachgewiesen und im Literatur- und Quellenverzeichnis aufgeführt. Das Gleiche gilt für graphische Darstellungen und Bilder sowie für alle Internet-Quellen.

Ich bin ferner damit einverstanden, dass meine Arbeit zum Zwecke eines Plagiatsabgleichs in elektronischer Form anonymisiert versendet und gespeichert werden kann.

Ort, Datum

Unterschrift des Verfassers / der Verfasserin

Veröffentlichung der Arbeit in der Bibliothek der Technischen Hochschule Aschaffenburg

Der Veröffentlichung der Bachelorarbeit in der Bibliothek der Technischen Hochschule Aschaffenburg wird zugestimmt.

Ort, Datum

Unterschrift des Verfassers / der Verfasserin

Extension of a RISC-V Core with Custom Matrix Multiplication instructions

Bachelorarbeit von
Wolfgang Bischoff

29. Januar 2026

Extension of a RISC-V Core with Custom Matrix Multiplication instructions

Extension of a RISC-V Core with Custom Matrix Multiplication instructions

Autor:
Wolfgang Bischoff
2228538

Erstprüfer:
Prof. Dr. Christian Jakob HDA



TECHNISCHE HOCHSCHULE ASCHAFFENBURG

FAKULTÄT INGENIEURSWISSENSCHAFTEN

WÜRZBURGER STRASSE 45

D-63743 ASCHAFFENBURG

Vorwort

Ich bedanke mich bei Herrn Prof. Dr. Christian Jakob für die Betreuung der Arbeit auf einem Themengebiet, dass mich persönlich sehr interessiert und viel Freude bereitet.

Ich bedanke mich bei der TH Aschaffenburg und allen beteiligten Mitarbeiterinnen und Professoren für die Organisation eines berufsbegleitenden Studiums. Ich schätze es sehr, dass es ermöglicht wird sich Chancen und Möglichkeiten zu erarbeiten, die man ohne den Studiengang nicht hätte.

Inhaltsverzeichnis

Abbildungsverzeichnis	IX
Glossar	X
1 Einleitung	1
1.1 Motivation	1
1.2 Aufgabenstellung	4
1.3 Struktur der Arbeit	4
2 Stand der Technik	6
3 Theoretische Grundlagen	8
3.1 Vektoren und Matrizen	8
3.2 Matrixmultiplikation mit SISD	9
3.3 Matrixmultiplikation mit Strip-Mining und SIMD	10
3.4 Die RISC-V Vector Extension	11
3.5 Blockweise Matrixmultiplikation	16
3.6 Outer-Produkt	17
4 Durchführung und Methodik	20
4.1 Der NEORV32 Core und dessen Einsatz	20
4.2 Test-Software	21
4.3 Evaluierung der Performance	23
4.4 Schrittweises Vorgehen	23
5 Analyse der NEORV32 CPU	25
5.1 Architektur des NEORV32 Core	25
5.2 Die Fetch-Engine	25
5.3 CPU-Control und Execution-Engine	26
5.4 Die Load-Store-Engine	27
6 Erweiterung der NEORV32-CPU	29
6.1 Die Matrix-Register	29
6.2 Die Instruktionen	29
6.3 Erweiterung der Execution-Engine	31
6.4 Erweiterung der Load-Store-Unit	35
6.5 Matrix-Engine	37
7 Auswertung und Evaluierung	41
7.1 Testdaten	41
7.2 Messung und Vergleichbarkeit	42
7.3 Prüfung mit Matrix-Erweiterung	44
7.4 Prüfung ohne Matrix-Erweiterung	46
7.5 Auswertung	48

8 Zusammenfassung und Ausblick	49
8.1 Zusammenfassung	49
8.2 Ausblick	50
Quellenverzeichnis	52

Abbildungsverzeichnis

1.1	View Frustum	2
1.2	Perspective Projection Matrix	2
1.3	Einfaches Neuronales Netz	3
1.4	Mehrere kombinierte Eingaben	3
3.1	Ansätze zur Matrix-Extension	15
3.2	Matrixmultiplikation blockweise	16
3.3	Matrixmultiplikation Ergebnis	17
3.4	Matrixmultiplikation Berechnungsvorschrift	17
3.5	Matrix Multiplikation Blöcke A, E, B und G	17
3.6	Matrix Accumulator (MAC)	19
5.1	NEORV32 Architektur	25
5.2	NEORV32 Gültige Instruktion	26
6.1	VHDL Entities	32
7.1	Steuersignale der Matrixmultiplikation	45
7.2	Waveforms der Matrixmultiplikation	46

Glossar

RVV

RISC-V Vector Extensions

SISD

Single Instruction Single Data

SIMD

Single Instruction Multiple Data

XLEN

Standardregisterlänge in Bits

VLEN

Vector-Registerlänge in Bits

MLEN

Matrix-Registerlänge in Bits

AVL Application Vector Length

SEW

Selected Element Width

LMUL

Group Modifier

VL Vector Length

IME Integrated Matrix Extension

VME

Vector Matrix Extension

AME

Attached Matrix Extension

MAC

Matrix Accumulator

CSR

Control and Status Register

LSU Load Store Unit

1 Einleitung

Mathematische Rechenvorschriften wie die Matrixmultiplikation existieren nicht zum Selbstzweck sondern finden in vielen Gebieten Anwendung. Dadurch sie zu einem extrem relevanten Leistungsmerkmal für moderne CPU Architekturen geworden. Die Anwendungsgebiete der Matrixmultiplikation sollen aufgezeigt werden.

Die Aufgabenstellung der Arbeit besteht in der Erweiterung einer RISC-V CPU um Anweisungen zur Berechnung der Matrixmultiplikation. Die Bedeutung von RISC-V und die Aufgabenstellung an sich wird beschrieben.

Das Kapitel wird mit einer Erläuterung zum Aufbau der Arbeit abgeschlossen.

1.1 Motivation

In dieser Arbeit soll ein RISC-V CPU um Anweisungen zur Durchführung von Matrixmultiplikationen erweitert werden.

RISC-V ist eine lizenzfreie Beschreibung eines Prozessors und der Anweisungen, die der Prozessor verarbeiten können muss. Durch diese Lizenzfreiheit ist die RISC-V Architektur sehr interessant sowohl für den akademischen als auch für den kommerziellen Einsatz.

Die Firma Qualcomm beispielsweise als eines der weltweit, führenden Unternehmen auf dem Gebiet der Halbleiter und der Telekommunikation hat eine RISC-V Erweiterung namens Xqci zu RISC-V und auch zu den Compiler-Framework LLVM beigesteuert [10]. Qualcomm führt auch Käufe von Firmen mit RISC-V Wissen durch. Dies deutet daraufhin, dass Qualcomm den Einsatz von RISC-V in Produkten ernsthaft in Erwägung zieht.

Matrixmultiplikation ist die Grundlage für perspektivische Projektionen, Rotationen und Bewegung von Objekten in der 3D Computer-Grafik. Dort ist es notwendig 4x4 Matrizen zu verwenden und mathematische Berechnungen so schnell wie möglich auf diesen Matrizen ausführen zu können.

In Abbildung 5.1 aus [1] wird auf der linken Seite das Sichtfeld (View Frustum) einer Kamera in einer Szene dargestellt. Durch die Matrix P_p wird dieses Sichtfeld perspektivisch korrekt auf den Einheitswürfel auf der rechten Seite abgebildet. Durch das Umformen des Sichtfelds erscheinen weiter entfernte Objekte z.B. kleiner. Diese Projektion muss auf jedes Objekt einer Szene angewendet werden. Mathematisch werden also viele Punkte mit einer Matrix

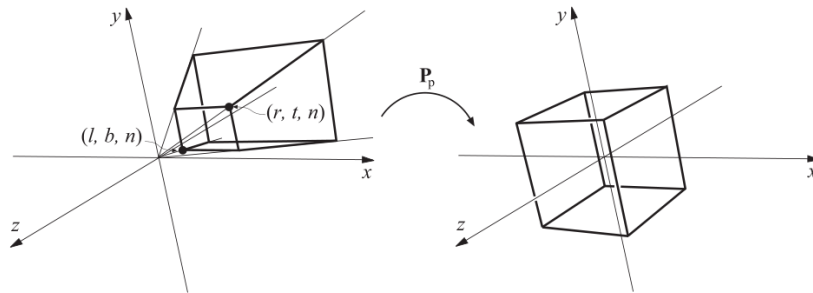


Abbildung 1.1: View Frustum

multipliziert um die transformierten Punkte zu erhalten. Diese zeigt, dass die Matrixmultiplikation extrem schnell ablaufen muss um eine hohe Bildwiederholrate zu erreichen. Die Projektionsmatrix ist in Abbildung 1.2 aus [1] dargestellt.

$$\mathbf{P}_p = \begin{pmatrix} \frac{2n}{r-l} & 0 & -\frac{r+l}{r-l} & 0 \\ 0 & \frac{2n}{t-b} & -\frac{t+b}{t-b} & 0 \\ 0 & 0 & \frac{f+n}{f-n} & -\frac{2fn}{f-n} \\ 0 & 0 & 1 & 0 \end{pmatrix}.$$

Abbildung 1.2: Perspective Projection Matrix

Der Einsatz von 3D-Computergrafik ist seit mehreren Jahren nicht mehr aus dem Alltag, aber auch aus vielen professionellen Bereichen nicht mehr wegzudenken.

Seit Google im Jahr 2017 die Transformer Architektur erfunden und OpenAI ChatGPT 1 programmiert hat, dauert die Erfolgsgeschichte von Large Language Models bis heute an. Generative künstliche Intelligenz wird als revolutionäre Erfindung betrachtet und führt schon jetzt dazu, dass Softwareentwickler Angst um ihren Wert auf dem Arbeitsmarkt haben. Es ist spätestens im Jahre 2026 klar, dass die KI Aufgaben bedeutend viel schneller erledigt als es ein Mensch jemals könnte.

Die rasante Verbreitung von LLMs hat Auswirkungen auf die Hardwarehersteller und den Handel der Hardware. Die Firma Micron hat beispielsweise einen Rückzug aus dem Endkundengeschäft vollzogen und verkauft RAM-Bausteine nur noch an Firmenkunden, die den RAM-Speicher dann bevorzugt für die Berechnung in Rechenzentren einsetzen. Es liegt nahe, dass LLMs in den Rechenzentren trainiert und betrieben werden. Die RAM-Preise für Endkunden sind aus diesem Grund innerhalb des Jahres 2025 um ein vielfaches angestiegen.

Gleiches gilt für Grafikkbeschleuniger der Firma NVIDIA. Auch hier konkuriert der Endkunde mit dem Firmenkunden. NVIDIA kann die Preise beinahe beliebig festlegen. Das Top-Produkt für Endkunden der Firma NVIDIA ist eine 5090 RTX Grafikkarte. Sie ist im Januar 2026 für etwa 3600 Euro erhältlich und kostet damit mehr als der gesamte Rest eines Computers. Eine Grafikkarte ist in der Lage Matrixmultiplikationen in großer Parallelität und extrem schnell zu berechnen. Grafikkarten werden auch zum Training von LLMs verwendet.

Ein LLM basiert auf Neuronalen Netzen. Zwei Neuronen in einem sehr einfachen Netz sind in Abbildung 1.3 aus [15] dargestellt.

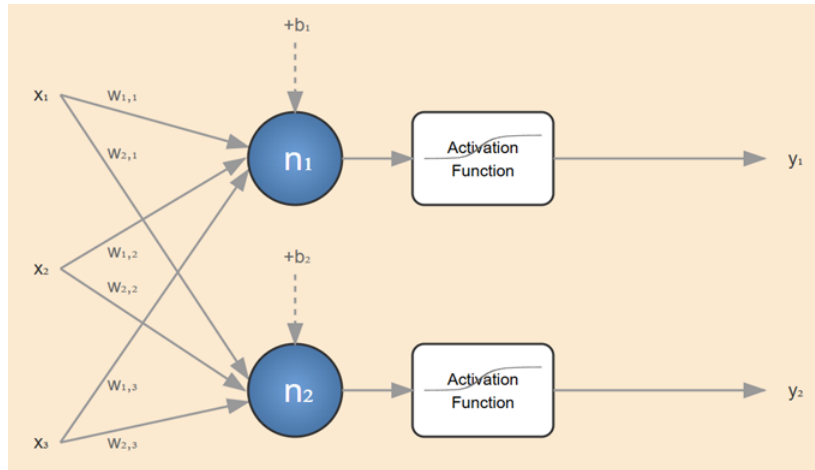


Abbildung 1.3: Einfaches Neuronales Netz

Eingaben an ein Neuronales Netz erfolgen in Form von Vektoren. Werden mehrere Eingaben x_i gleichzeitig an das neuronale Netz gegeben, wird aus der Vektorrechnung ein Matrix-Rechnung, da gleichzeitig mit mehreren Gewichtungen $w_{i,j}$ multipliziert werden muss. Dieser Umstand ist in 1.4 aus [15] dargestellt

$$\begin{bmatrix} w_{1,1} & w_{1,2} & w_{1,3} \\ w_{2,1} & w_{2,2} & w_{2,3} \end{bmatrix} \begin{bmatrix} x_1^1 & x_1^2 \\ x_2^1 & x_2^2 \\ x_3^1 & x_3^2 \end{bmatrix} = \begin{bmatrix} w_{1,1} \cdot x_1^1 + w_{1,2} \cdot x_2^1 + w_{1,3} \cdot x_3^1 & w_{1,1} \cdot x_1^2 + w_{1,2} \cdot x_2^2 + w_{1,3} \cdot x_3^2 \\ w_{2,1} \cdot x_1^1 + w_{2,2} \cdot x_2^1 + w_{2,3} \cdot x_3^1 & w_{2,1} \cdot x_1^2 + w_{2,2} \cdot x_2^2 + w_{2,3} \cdot x_3^2 \end{bmatrix}$$

Abbildung 1.4: Mehrere kombinierte Eingaben

Getrieben durch den Erfolg von LLM kann sich eine Spezifikation für CPUs, wie z.B. RISC-V nicht mehr durchsetzen, wenn Sie keine Anweisungen für den effizienten Umgang mit Vektoren und Matrizen besitzt. Die Endkunden werden Produkte, die nicht in der Lage sind Matrizen hardwarebeschleunigt zu berechnen nicht mehr kaufen z.B. aus Gründen der Computergrafik. Das gleiche gilt für Firmenkunden, die den Betrieb eines LLM planen. Ohne beschleunigte Matrixberechnung kann sich ein CPU nicht mehr auf dem Markt behaupten.

Die CPUs der Firmen AMD und Intel besitzen bereits die Erweiterungen AVX2 (Advanced Vector Extensions 2), AMX (Advanced Matrix Extensions) und FMA (Fused Multiply Add) zum Rechnen mit Vektoren und Matrizen.

All diese Punkte zusammengekommen ergeben eine große Motivation für den Einsatz von RISC-V und den Einsatz von Matrixmultiplikation innerhalb einer RISC-V CPU.

1.2 Aufgabenstellung

Die NEORV32 RISC-V CPU wird um Anweisungen erweitert, die die hardwarebeschleunigte Matrixmultiplikation ermöglichen. Dabei orientieren sich die Anweisungen von der Kleinteiligkeit her an den Anweisungen, wie sie bereits für die RVV-Vektor-Extension ratifiziert worden sind. Im speziellen wird das Konzept des Strip-Mining ermöglicht. Dabei handelt es sich um die Aufteilung des Problems in Teile, die die CPU, aufgrund von begrenzter Ressourcen, schrittweise zugeführt werden, bis die Aufgabe als ganzes gelöst ist.

Die Erweiterung soll mit dem Wissenstand eines Beginners in der Hardware-Beschreibung vorgenommen werden können. Es wird im speziellen eine Erweiterung des bereits existierenden NEORV32 Core vorgenommen.

Die Matrizenberechnungen erfolgen im ersten Schritt auf Matrizen deren Dimensionen ein Vielfaches der Zahl vier sind (also z.B. 4x4, 8x8, 16x16) und es werden nur die ganzzahlige Multiplikationen ohne Vorzeichen durchgeführt. Ohne Beschränkung der Allgemeinheit lässt sich eine Matrix an den Rändern mit Nullen auffüllen um auf ein Vielfaches der Dimension vier zu kommen, ohne das Ergebnis der Multiplikation zu verändern.

Da es viele Ansätze gibt Matrizen zu multiplizieren, wurde eine Einschränkung getroffen. Die Multiplikation erfolgt mit dem Outer-Produkt Ansatz, da dieser Ansatz auch in dem maßgebenden Dokument zu dieser Arbeit beschrieben ist [9].

1.3 Struktur der Arbeit

Es wird mit einer Erfassung des Status Quo begonnen. Der Stand der Technik wird beschrieben.

Als nächstes wird die Matrixmultiplikation im Kapitel "Theoretische Grundlagen" beschrieben. Es wird beschrieben, dass sich die Matrixmultiplikation in Teile aufteilen und damit schrittweise abarbeiten lässt.

Danach werden die momentan ablaufenden Anstrengungen des RISC-V Gremiums zur Erstellung einer Extension zur Matrixmultiplikation beschrieben. Dabei gibt es drei Ansätze. Der in dieser Bachelorarbeit untersuchte Ansatz wird zusammen mit den anderen Ansätzen vorgestellt. Dabei wird der Strip-Mining Ansatz erläutert und es wird eine Softwareanwendung besprochen, die diesen Ansatz als Computerprogramm implementiert.

Es folgt eine Beschreibung des Vorgehens bei der Erstellung dieser Bachelor-Arbeit.

Nachdem der Ansatz beschrieben ist, wird die NEORV32 CPU als das Ziel der Implementierung beschrieben. Das Design wird herausgearbeitet und es wird diskutiert, welche Änderungen an welchen Stellen vorgenommen werden, um die CPU um Matrixbefehle zu erweitern.

Danach wird der VHDL Quellcode der NEORV32 CPU erweitert und auf einen FPGA programmiert. Auf dem FPGA wird eine Software-Anwendung ausgeführt, die die Matrix-Instruktionen verwendet um eine Matrixmultiplikation durchzuführen.

Schließlich wird das Ergebnis mit einer normalen Matrixberechnung verglichen.

Die Arbeit endet mit einer Zusammenfassung und einem Ausblick auf mögliche Verbesserungen.

2 Stand der Technik

Ein RISC-V Komitee hat im Jahre 2015 mit der ersten Version der RISC-V Vector Extension (RVV) begonnen. Im Jahre 2021 wurde das Release 1.0 ratifiziert und ist jetzt in einen finalen Zustand (Frozen) überführt worden. RISC-V wird durch die (RVV) um Register für Vektoren und explizite Anweisungen zur Manipulation dieser Vektoren erweitert.

In der RISC-V Vector Extension Spezifikation [2] wird das Konzept des Strip-Mining beschrieben und hervorgehoben. Die Software lädt den Cache mit einem Teil einer oder mehrerer Vektoren voll und beginnt den Cache dann abzuarbeiten, in dem der Teil der Vektoren dann wiederum teilweise in den CPU geladen, verarbeitet und danach wieder aus dem CPU zurück in den Speicher übertragen wird. Das Strip-Mining wiederholt diese Abläufe, bis die Aufgabe erledigt ist, also alle Element der Vektoren bearbeitet sind. Das Beispiel in Kapitel 6.4. namens "Example of stripmining and changes to SEW" aus [2] verdeutlicht dieses Vorgehen.

Die Alternative zum Hardware unterstützen Strip-Mining wäre es, jedes einzelne Vektorelement einzeln in ein Register der CPU zu laden und dann mit einem Element des zweiten Vektors zu verrechnen. Dabei wird für jedes Element ein Laden und Speicher aus dem Speicher notwendig und man bezahlt mit einem hohen Verlust an Geschwindigkeit für die im Gegenzug triviale Implementierung.

Der Vorteil der RVV Register ist es, dass diese Vektorregister mehrere Elemente parallel auf eine Instruktion hin zusammenrechnen können. Der CPU verfährt dann in einem echten SIMD-Betriebsmodus (Single Instruction Multiple Data) und spart ein vielfaches an Taktzyklen im Vergleich zur sequentiellen Ausführung.

Das Prinzip des Strip-Mining wird auch in mathematischen Bibliotheken verwendet. Eine bekannte Bibliothek ist blis [17]. blis verwendet den Begriff Kernel oder Microkernel. Ein Kernel ist Code, der auf einem Cache-Frame ausgeführt wird, also den Code darstellt, den die CPU sehr schnell ausführen kann. Optimierte Anweisung für verschiedene CPU Architekturen werden hauptsächlich in dem Code verwendet, der dem Kernel zuzurechnen ist um Zeit zu sparen.

Um die Berechnungen im Kernel herum gibt es weiteren Code, der über Schleifen nacheinander alle Daten in den Kernel lädt, bis die Berechnung vollständig abgeschlossen ist.

Es gibt sehr starke Parallelen zwischen dem Strip-Mining und dem Kernel-Ansatz der blis Bibliothek. Sollte blis für RISC-V portiert werden, ist es sinn-

voll, dass RISC-V das Prinzip des Strip-Mining ermöglicht. Dies wurde durch die RISC-V Vector Extension für Vektoren bereits erreicht.

Für die RISC-V Matrixberechnung gibt es zum jetzigen Zeitpunkt keine ratifizierte Extension. Innerhalb der RISC-V Arbeitsgruppen werden unterschiedliche Ansätze diskutiert. Die Ansätze werden im Kapitel zu den theoretischen Grundlagen beschrieben.

Interessanterweise gibt es eine chinesische Firma namens Stream Computing die sich zur Mission gesetzt hat, high-end RISC-V Chips zum Einsatz in Rechenzentren zu entwickeln. Bereits im Jahre 2022 konnte Stream Computing ein eigenes Arbeitsergebnis vorweisen. Dieses Arbeitsergebnis wurde von Stream Computing selbst "RISC-V Matrix ISA Specification" in der Version v0.1 genannt. Die Version v0.5 wurde im Oktober 2024 erstellt. Es handelt sich aber trotz des Namens nicht um die offizielle Version, die das RISC-V Gremium veröffentlichen wird! Stream Computing hat ebenfalls die notwendigen Anpassungen zum LLVM Compiler-Framework beigetragen!

Wie tragbar die Ergebnisse sind lässt sich schwer einschätzen ohne eine eingehende Evaluierung des Quellcodes durchgeführt zu haben. Es ist jedoch interessant zu sehen, welche Innovationskraft und Arbeitsgeschwindigkeit die chinesischen Industrie hat. Während sich das RISC-V Gremium wieder auf eine mehrjährige Reise begibt um eine Matrix-Extension zu erzeugen, wie es bereits bei der Vektor-Extension erfolgt ist, hat eine chinesische Firma vermeintlich bereits ein Design und eine Toolchain zu diesem Design erstellt.

Innerhalb des NEORV32 CPU Umfeldes gibt es eine Arbeit von Qizal Arsalan [3] bei der der NEORV32 CPU um Matrixberechnungen erweitert wird. Diese Bachelorarbeit unterscheidet sich allerdings hinsichtlich der Herangehensweise stark von Qizal Arsalans Arbeit. Der NEORV32 CPU besitzt im Design vorgesehene Erweiterungspunkte namens CFS und CFU. Das CFS (Custom Functions Subsystem) erlaubt es Hardware-Beschleuniger in den NEORV32 CPU einzubringen, die über Speicheradressen (Memory Mapping) aus einer Softwareanwendung mit Daten versorgt werden können. CFU (Custom Function Unit) erlaubt es einzelne Instruktionen zu ergänzen um den CPU um kleinschrittige Operationen zu erweitern.

Qizal Arsalan untersucht den Ansatz der Matrixmultiplikation mit CFS und CFU. In dieser Bachelorarbeit werden Matrix-Instruktionen in den NEORV32 Chip eingebracht, die nicht über das CFS oder CFU Feature eingebunden werden sondern über die Execution-Engine, also der State-Machine, des NEORV32 Chips selbst. Der Grund ist, dass die Speicheradressabhängigkeit von CFS und CFU nicht konform mit einer Matrix-Extension-Spezifikation des RISC-V Gremiums sein können und nicht sein werden.

Bei CFS und CFU handelt es sich um Systeme, die nicht Teil der RISC-V Spezifikation sind. Diese Bachelorarbeit versucht die Ergebnisse einer Matrix-Extension im RISC-V Umfeld zu antizipieren und kann daher CFS und CFU nicht verwenden.

3 Theoretische Grundlagen

Dieses Kapitel beschreibt Vektoren und Matrizen aus der Mathematik und deren Berechnung durch ein Computersystem.

Bei der Berechnung ist auf die Geschwindigkeit zu achten. Ausgehend von dem naiven Berechnungsverfahren werden die Schwächen dieses Verfahrens diskutiert und beschrieben, welche Antworten das RISC-V Ökosystem auf diese Probleme hat.

Die ersten Hardwarebeschleunigungen sind mit der RISC-V Vector Extension für Vektoren eingeführt worden. Damit kann ein RISC-V Vektor SIMD unterstützen. Aufbauend auf der Vector-Extension wird diskutiert, welche Ansätze für die Berechnung von Matrizen verwendet werden können.

Der in dieser Arbeit gewählte Ansatz wird beschrieben.

3.1 Vektoren und Matrizen

Ein Vektor ist eine eindimensionale Gruppierung von skalaren Werten. Es handelt sich dabei um eine Fortführung von Skalaren zu neuen Objekten der Mathematik, wie sie auch z.B. bei komplexen Zahlen stattfindet.

Auf Vektoren sind mathematische Operationen wie Addition definiert, die wiederum Vektoren erzeugen. Es sind auch Operationen wie das Skalarprodukt oder der Betrag definiert die aus Vektoren wieder Skalare erzeugen.

Eine Matrix wird in [16] als rechteckiges Zahlenschemata mit m Zeilen und n Spalten beschrieben.

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & \dots & a_{1n} \\ a_{21} & a_{22} & a_{23} & \dots & a_{2n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{d1} & a_{d2} & a_{d3} & \dots & a_{dn} \end{bmatrix}$$

Die Multiplikation zwischen zwei Matrizen A und B ist nicht kommutativ und nur definiert, wenn die inneren Dimensionen der Matrizen übereinstimmen, wenn also die Spaltenzahl der ersten Matrix mit der Zeilenzahl der zweiten Matrix übereinstimmt.

Zur Berechnung der Matrixmultiplikation

$$C = A * B \quad (3.1)$$

muss zur Berechnung einer Zelle der Matrix C ein Skalarprodukt aus einer Zeile von A und einer Spalte von B berechnet werden. Dabei wird zunächst aus der Spalte von B durch Transponieren eine Zeile gemacht und dann der Zeilenvektor aus A mit dem Zeilenvektor aus B durch das Skalarprodukt in einen Skalar umgerechnet. Dieser Skalar wird dann in die Matrix C eingesetzt.

Ein anschaulichere Beschreibung bietet das Falksche Schema [6].

3.2 Matrixmultiplikation mit SISD

Eine Implementierung, die sich direkt an der mathematischen Rechenvorschrift orientiert, verwendet drei For-Schleifen.

```
1 C = zero_matrix() // Starte mit Nullwerten
2
3 for int i = 1 to l // ueber die Zeilen von C
4   for int k = 1 to n // ueber die Spalten von C
5     for int j = 1 to m // ueber die Spalten von A
6       // und Zeilen von B
7       C(i,k) = C(i,k) + A(i,j) * B(j,k)
8     end
9   end
10 end
```

Der Nachteil dieser Beschreibung ist nicht sichtbar, wenn man sich den Pseudo-Code oder eine Implementierung in einer realen, höheren Programmiersprache ansieht. Diese Implementierung beschränkt sich auf SISD (Single Instruction Single Data) Anweisungen. Wenn der Compiler das Optimierungspotential nicht erkennt, generiert er SISD Anweisungen, die jede Multiplikation und Addition einzeln ausführen.

In den beiden Architekturen Van-Neumann und Harvard gilt gleichermaßen, dass eine Berechnung nicht im Speicher sondern nur innerhalb der CPU ausgeführt wird. Da die Daten im Speicher stehen muss also eine Übertragung zwischen Speicher und CPU erfolgen. Man nennt dies die Load-Store-Architektur.

Der Nachteil der naiven Implementierung liegt also in der Tatsache begründet, dass Werte nicht in den Registern der CPU verbleiben können, da eine CPU nur wenige Register besitzt und diese Register wiederverwendet werden müssen. Um die Register wiederzuverwenden muss der enthaltene Wert zurück in den Speicher geschrieben werden! Dies ist in der höheren Programmiersprache nicht dargestellt und wird erst sichtbar, wenn man sich den Assembler-Quellcode ansieht, den der Compiler erstellt.

Im aller ungünstigsten Fall besteht dieser Assembler-Code aus einer langen Folge aus einer Load-Anweisung gefolgt von einer Operation gefolgt von einer Store-Anweisung. Dieses ständige Laden und Speichern führt zu einer langen Laufzeit.

3.3 Matrixmultiplikation mit Strip-Mining und SIMD

Ein Ansatz diesen Verlust, der durch Load-Store entsteht zu minimieren, ist das Strip-Mining in Kombination mit der Verwendung von SIMD. SIMD (Single Instruction Multiple Data) erlaubt es mehr als ein Datum gleichzeitig durch eine einzige Instruktion verarbeiten zu können. Die Addition zweier Vektoren geschieht elementweise und jedes Elementpaar kann unabhängig von allen anderen Paaren in jeglicher Reihenfolge addiert werden. Damit eignet sich die Vektoraddition für SIMD Anweisungen, wenn die CPU die passende Hardware bereitstellt.

Des weiteren erlaubt das Strip-Mining auch Datenobjekte zu verarbeiten, die so groß sind, dass sie nicht als Ganzes in die Register der CPU passen.

Beim Strip-Mining wird von einer Ressourcen-Hierarchie ausgegangen. Die Ressourcen sind dabei die Register und der Speicher einer CPU, wobei Register letztendlich auch nur sehr kleine Speicherzellen sind. Der Grundgedanke ist, dass je näher Speicher am CPU liegt, das dessen Preis steigt und daher kleiner in der verfügbaren Größe ausfallen muss, aber gleichzeitig auch um einiges schneller als ein weiter entfernt liegender Speicher ist. Die Hierarchie besteht aus den Registern, gefolgt von einem oder mehreren Cache-Leveln und schließlich den RAM-Bausteinen.

Die Vektor- oder Matrix-Daten kommen aus dem Speicher und müssen dem CPU in Form seiner Register zugeführt werden. Die Addition zweier Vektoren Beispielsweise kann nur durchgeführt werden, wenn Vektordaten in den Registern der CPU stehen. Die Kunst besteht also darin, die Daten in effizienter Form an den CPU heranzutragen. Beispielsweise ist es ineffizient ständig im Speicher hin- und her zu springen. Ein lineares Laden ist effizienter um die Lokalität der Daten innerhalb des Cache auszunutzen.

Eine Cache-Hierarchy legt nahe, Vektoren blockweise zu laden, so das ein Block jeweils genau in den verfügbaren Cache passt. Einmal geladen, dient der Cache dem CPU als schnellen Speicher und ein Durchgriff auf den langsameren RAM-Speicher erfolgt nur, wenn die Daten im Cache verarbeitet worden sind.

Die Register innerhalb der CPU sind von begrenzter Größe. Eine sinnvolle Annahme ist etwa 1024 bit Registerbreite. Damit passen dort dann 32 Elemente eines Vektors hinein, wenn jedes Element eine 32-Bit Zahl darstellt.

Wenn nun sehr große Vektoren addiert werden sollen, dann kommt Strip-Mining zum Einsatz. Mit groß ist hier gemeint, dass die Vektoren weder im Ganzen in die CPU-Register passen noch passen die Vektoren im Ganzen

in einen Cache-Frame. Die Software verwendet eine Instruktion um zu ermitteln, wie viel Platz innerhalb der CPU-internen Register verfügbar ist. Die CPU antwortet mit der Registerbreite.

Für die Berechnung von Vektoren wurde die RISC-V Vector Extension (RVV) [2] erstellt und ratifiziert. Die Vektor Extension ist für diese Arbeit relevant, obwohl sie zwar Vektoren verarbeitet aber damit die Grundlage für die Verarbeitung von Matrizen legt.

3.4 Die RISC-V Vector Extension

Ein RISC-V CPU besitzt die sogenannte X-Register-File. Es handelt sich damit um alle Register, die ein RISC-V CPU standardmäßig, ohne Extensions hat. Dies könnten beispielsweise die Register x0 bis x31 sein. Die Länge der Register in bits wird durch die Konstante XLEN festgelegt.

Die RVV erweitert einen RISC-V CPU um zusätzliche Vektor Register v0 bis v31. Die Länge der Vektor Register wird durch die Konstante VLEN (in Anlehnung an die Konstante XLEN) vorgegeben. VLEN wird bei der Synthese des Designs auf einen Wert festgelegt und ist nicht zur Laufzeit änderbar. VLEN stellt somit eine Eigenschaft der CPU dar. Die RVV Spezifikation [2] macht die Vorgabe:

“The number of bits in a single vector register, $VLEN \geq ELEN$, which must be a power of 2, and must be no greater than 2^{16} ”.

Das besondere an den Vector Registern ist, dass sie konfiguriert werden können. Es lässt sich festlegen, welcher Datentyp in den Registern erwartet wird, also 8-bit Bytes, 16-bit Halbwörter oder 32- oder 64-bit Wörter. Zusätzlich lassen sich die Vektorregister Gruppieren, also aneinanderhängen um noch größere Register zu bilden.

Der Vorteil ist, dass man beispielsweise 8-bit bytes als Datentyp festlegen kann und dann vier Vektorregister gruppieren kann. Unter der Annahme das ein RISC-V Chip mit einer VLEN von 2^{16} synthetisiert worden ist, ergibt sich damit Speicher für beispielsweise 262144 bytes. Werden zwei Vektoren dieser Länge in den CPU geladen, kann eine Vektoraddition all dieser Elemente mit nur einer einzigen Instruktion erfolgen! Dies ergibt einen Speedup um den Faktor 262144 gegenüber einem naiven Load-Store Ansatz, bei dem jedes 8-Bit Elementpaar einzeln geladen und verrechnet wird.

Die Verwendung der Vector-Extension erfolgt durch ein Hardware-Software-Co-Design. Damit ist gemeint, dass eine Anwendung mit der Hardware zusammenarbeiten muss um von der Hardware-Beschleunigung zu profitieren. Es ist weder möglich durch Software oder Hardware alleine eine Beschleunigung zu erzielen.

Moderne, optimierende Compiler wie der GCC oder die LLVM Infrastruktur wurden um das Wissen um die RISC-V Vektor-Extension erweitert und

können den Anwendungsfall automatisch erkennen und das gewünschte Hardware-Software Co-Design automatisch generieren!

Dies lässt sich testen indem man den naiven Matrix Multiplikationsalgorithmus mit der neuesten Version des GCC für RISC-V übersetzt. Im generierten Assembler-Code findet man die RVV-Instruktionen wieder! Der Compiler hat den Anwendungsfall erkannt und das Co-Design erstellt.

Das Hardware-Software Co-Design sieht das iterative Vorgehen Strip-Mining vor, bei dem die Software die Hardware in jeder Iteration aufs neue konfiguriert und dann Daten zur parallelen Verarbeitung durchführt.

Die Schritte sind:

1. Die erste bzw. nächste Iteration beginnt.
2. Die Applikation entscheidet, ob es effizienter ist die Vektoroperation mit oder ohne Vektor-Extension auszuführen. Wenn die Vektorlänge beispielsweise nur aus zwei Elementen besteht ist es schneller diese beiden Element einfach so zu verrechnen.
3. Wenn sich die Anwendung dazu entscheidet die Vektor-Extension zu verwenden, dann beginnt die Strip-Mining-Schleife. Wenn es zu wenige Elemente gibt, dann springt die Anwendung zu einem Label, dass die Verarbeitung terminiert und berechnet den Rest der Element dort manuell ohne Hardwarebeschleunigungen.
4. Die Anwendung bestimmt den Anteil der Vektorelemente, die noch verarbeitet werden müssen. Dieser Wert wird Application Vector Length (AVL) genannt. Beispielsweise entspricht die AVL in der ersten Iteration der gesamten Länge des Vektors. Mit jedem Schleifendurchlauf werden Teile des Vektors verarbeitet und die AVL nimmt immer mehr ab bis schließlich keine Elemente zur Verarbeitung mehr übrig sind und das Verfahren terminiert.
5. Neben der AVL muss die Anwendung auch in jeder Iteration den Datentyp bzw. die Größe der einzelnen Vektorelemente konfigurieren. Die Größe wird (= Selected Element Width, SEW) genannt. Es wird auch bestimmt, wie die Vektorelement in den Registern angeordnet werden und wie die Vektorregister gruppiert werden. Diese Konfiguration wird Group Modifier oder LMUL genannt. Um die Konfiguration vorzunehmen ruft die Anwendung die Instruktion `set(i)vl` auf.
6. Die Vector Engine verwendet die konfigurierten Werte AVL, SEW und LMUL und berechnet die Batch Größe. Ein Batch ist die Menge an Vektorelementen die in einer einzigen SIMD-Operation verrechnet werden. Hier kommt der Hardware-Teil des hardware-software co-designs zum tragen, denn der RISC-V CPU und die enthaltene Vector-Engine müssen entscheiden, wie viele Elemente jetzt tatsächlich parallel verarbeitet werden können. Dabei berücksichtigt die Vector-Engine wie viele Rechenknoten sie zur Verfügung hat um parallel zu rechnen. Die ermittelte Anzahl an Elementen wird vector length (VL) genannt und von der

Hardware an die Anwendung zurückgegeben. Dazu gibt die Instruktion `set(i)vli` diesen Wert in ein Rückgaberegister aus, aus dem sich die Anwendung den Wert holen kann.

7. Die Anwendung kennt jetzt die VL, die für diese Iteration gilt. Damit kennt die Software die Batch-Size. Die Software wird nun die Vector-Load Instruktionen (`vle8`, `vle16`, ...) verwenden um einen Anteil des Vektors oder der Vektoren, der genau der VL entspricht in die Vektorregister zu laden. Die Anwendung wird auch einen Zeiger nach vorne verschieben, um sich zu merken von welcher Stelle in der nächsten Iteration geladen werden muss.
8. Die Vector-Engine verarbeitet die Vector-Load Anweisungen und schreibt die geladenen Daten in die Vektorregister.
9. Die Anwendung führt nun eine Instruktion aus um die Vektoren zu verrechnen. Es können beispielsweise eine elementweise Addition (`vadd`), Subtraktion (`vsub`), Multiplikation oder sonstige Operationen ausgeführt werden.
10. Die Vektor-Engine führt nun die Instruktion auf allen Elementen parallel aus. Es handelt sich also um eine SIMD-Operation, bei der mehrere Daten aufgrund einer einzigen Anweisung verarbeitet werden. Damit ist der aktuelle Batch der Iteration abgearbeitet.
11. Die Anwendung führt eine Store-Anweisung aus um die Ergebnisdaten aus dem Ergebnisregister in den Speicher zurückzuschreiben und Platz für neue Ergebnisdaten zu schaffen.
12. Die Vektor-Engine verarbeitet die Store-Anweisung und die Daten werden in den Cache bzw. den RAM übertragen.
13. Die nächste Iteration beginnt. Im Grunde springt die Anwendung von hier zurück zum ersten Punkt dieser Auflistung.

Es folgt ein Beispiel in RISC-V Assembly, das das obige Vorgehen umsetzt.

```
1 // vector-vector add routine of 32-bit integers
2 //
3 // void vvaddint32(size_t n,
4 //     const int*x, const int*y, int*z)
5 // {
6 //     for (size_t i=0; i<n; i++)
7 //     {
8 //         z[i] = x[i] + y[i];
9 //     }
10 // }
11 //
12 // Mapping parameters to argument register (a0 .. a7)
13 // a0 = n, a1 = x, a2 = y, a3 = z
14 //
15 // Non-vector instructions are indented
16 vvaddint32:
17
18     // Set vector length based on 32-bit vectors.
```

```
19 // t0 contains the amount of elements that will be
20 // processed
21 vsetvli t0, a0, e32, ta, ma
22
23 // Load for first vector
24 vle32.v v0, (a1)
25
26 // Decrement number of elements that
27 // still need processing.
28 // t0 is the result of vsetvli. It
29 // is the amount of elements processed
30 // this iteration
31 sub a0, a0, t0
32
33 // Multiply number done by 4 bytes (= 32 bit
34 // integer) because of SEW=e32
35 slli t0, t0, 2
36
37 // Increment/Advance pointer for first vector
38 add a1, a1, t0
39
40 // Load for second vector
41 vle32.v v1, (a2)
42 // Increment/Advance pointer for second vector
43 add a2, a2, t0
44
45 // Sum vectors
46 vadd.vv v2, v0, v1
47
48 // Store result back to memory
49 vse32.v v2, (a3)
50
51 // Increment/Advance pointer for destination
52 add a3, a3, t0
53
54 // Loop back to the beginning of the for loop
55 // if the element count is not zero. Otherwise
56 // continue with the ret instruction
57 bnez a0, vvaddint32
58
59 // Finished with the function
60 ret
```

Jetzt, da die Wirkungsweise der RVV Extension beschrieben worden ist, kann nachvollzogen werden, in welchem Spannungsfeld die RISC-V Gremien Entscheidungen treffen müssen, wenn es um den nächsten Schritt hin zu einer Matrix-Extension geht.

Da die RVV Extension ratifiziert worden ist, gibt es bereits Chips die auf dem Markt verfügbar sind, und die die RVV implementieren. Ein Beispiel ist der OrangePi RV2 Single Board Computer, der einen Spacemit-K1 RISC-V Chip verwendet.

Das bedeutet, Firmen haben bereits Designs erstellt und Investitionen in die RISC-V RVV Strategie investiert. Wenn eine Matrix-Extension Vektorregister wiederverwenden würde, hätten solche Firmen einen Vorteil. Auf der anderen Seite möchte man das bestmögliche Verfahren ermöglichen um effizient mit Matrizen rechnen zu können und sich damit von der Konkurrenz abzuheben. Eventuell ist der Ansatz, der für Vektoren gewählt wurde, für Matrizen nicht optimal.

Es gibt daher laut Krste Asanovic [4] bei der Arbeit an einer Extension für Matrizen drei große Strömungen.

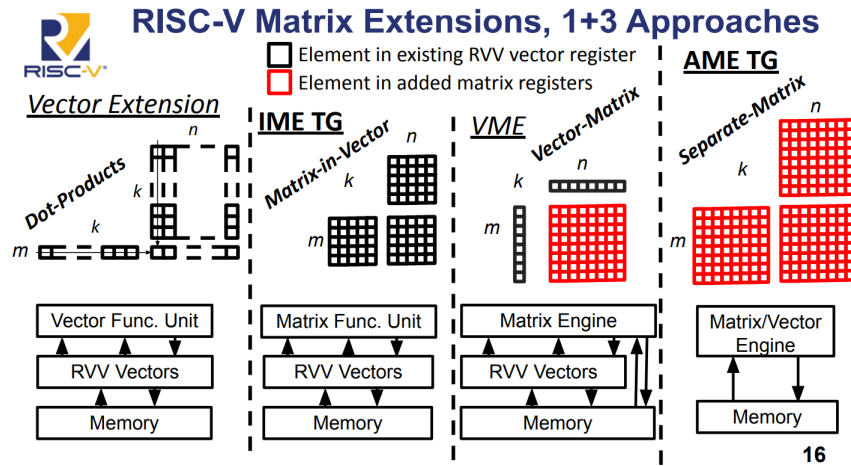


Abbildung 3.1: Ansätze zur Matrix-Extension

Zunächst wird in der linken Spalte der Abbildung 3.4 gezeigt, dass sich eine Matrix theoretisch elementweise berechnen lässt, wenn für jedes Element der Matrix individuell eine Dot-Produkt mit Hilfe der RVV-Extension berechnet wird. Dies ist nur möglich, wenn die CPU die RVV besitzt und gleichzeitig, wenn die Matrix spalten- oder zeilenweise in ein Vektorregister passt.

Die Integrated Matrix Extension IME [8] verwendet die RVV-Register wieder um darin Matrix-Berechnungen auszuführen. Es wird keine spezielle Hardware für die Matrix-Berechnung hinzugefügt. Die erklärten Ziele des IME-Charter sind es, einen bedeutenden Geschwindigkeitsvorteil zu erzielen, bei vergleichbaren Kosten, wie sie bereits für die RVV anfallen.

Die Vector-Matrix Extension VME baut auf die RVV-Extension auf und führt zusätzlich einen sogenannten Matrix-State bzw. eine Matrix-Engine hinzu. Mit dem Matrix-State ist gemeint, dass die CPU um Komponenten erweitert wird, die in der Lage sind Zwischenergebnisse bei einer Matrix-Multiplikation zu speichern. Es kommt bei der VME also weitere Hardware für die Matrix-Berechnung hinzu. Diese zusätzliche Hardware ist in der Abbildung in roter Farbe markiert.

Die Attached Matrix Extension AME geht noch einen Schritt weiter und verwendet dabei keine Vektorregister aus der RVV-Extension mehr sondern fügt Architected State sogar für komplette Matrix Register hinzu. Sowohl die Eingabedaten als auch die Zwischen- und Endergebnisse werden in speziellen Matrix-Registern gespeichert. Damit ist die AME als einziger Ansatz unabhängig von der RVV-Extension und könnte auch komplett ohne RVV-Extension angeboten werden. Auch hier ist neue Hardware in roter Farbe dargestellt.

Für die offizielle Ratifizierung einer RISC-V Matrix-Extension ist zum jetzigen Zeitpunkt nicht klar, welcher Ansatz verwendet wird oder ob eventuel sogar mehr als eine Spezifikation ratifiziert werden wird.

In dieser Bachelor-Arbeit wird ein Ansatz verfolgt, der sich vom Architected State her am ehesten an der Attached Matrix Extension AME orientiert. Zusätzlich werden Instruktionen vorgesehen, wie sie bei der Vektor-Extension zu sehen sind. Damit ist gemeint, dass die hinzugefügten Instruktionen den Strip-Mining Ansatz ermöglichen. Damit kann eine Matrixmultiplikation auch für sehr große Matrixen, die nicht als Ganzes in die CPU-Register passen, erfolgen. Außerdem können so auch Kernel in der blis Bibliothek implementiert werden.

3.5 Blockweise Matrixmultiplikation

Im Folgenden wird das blockweise Vorgehen erläutert, mit dem die Matrixmultiplikation ausgeführt werden wird.

Bei der herkömmlichen Matrixmultiplikation über das Dot-Product von Zeilen und Spalten werden die beiden Matrizen A und B in ihre komplette Dimension betrachtet, da jeweils komplette Zeilen und Spalten verrechnet werden um ein Matrix-Element zu bestimmen.

Bei sehr großen Matrizen ist dies nicht zielführend, da die CPU die Matrix nur schrittweise erfassen könnte und damit nur sehr ineffizient berechnen kann.

Die angestrebte Verbesserung besteht darin, eine Matrix blockweise zu berechnen. Damit eröffnet sich, ähnlich wie bei Vektoren, das Potential der parallelen Berechnung durch SIMD Instruktionen. Wie bei Vektoren ist es durch das blockweise Vorgehen auch für die Berechnung von Matrizen egal in welcher Reihenfolge die einzelnen Ergebnisse berechnet werden. Damit ist auch eine vollkommene Parallelität denkbar.

Als Ausblick würde das für ein RISC-V Design bedeuten, dass ein RISC-V CPU aus mehreren Harts synthetisiert wird, wobei jeder Hart eine Matrix-Engine erhält, also in der Lage ist hardwarebeschleunigt die Matrixmultiplikation zu berechnen. Dann wird eine große Matrix auf alle Cores verteilt um das Ergebnis in einzelnen Blöcken zu berechnen und zurück in den RAM-Speicher zu schreiben.

Zunächst wird anhand eines Beispiels verdeutlicht, was es bedeutet, wenn von blockweiser Berechnung gesprochen wird.

A	B	E	F
C	D	G	H
1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16
17	18	19	20
21	22	23	24
25	26	27	28
29	30	31	32

Abbildung 3.2: Matrixmultiplikation blockweise

In Abbildung 3.2 wird die grüne Matrix mit der blauen Matrix multipliziert. Es werden einzelne Blöcke A bis H innerhalb der beiden Matrizen definiert.

Zur Gegenprüfung ist das Ergebnis in Abbildung 3.3 dargestellt.

250	260	270	280
618	644	670	696
986	1028	1070	1112
1354	1412	1470	1528

Abbildung 3.3: Matrixmultiplikation Ergebnis

Dieses Ergebnis wird den Berechnungsvorschriften aus 3.4 zufolge berechnet. Beispielsweise gilt für den grauen Block die Vorschrift $A * E + B * G$

A	B	E	F	$A * E + B * G$	$A * F + B * H$
C	D	G	H	$C * E + D * G$	$C * F + D * H$
nach links		nach unten			

Abbildung 3.4: Matrixmultiplikation Berechnungsvorschrift

Der grau markierte Block ist das Ergebnis der Multiplikation der Blöcke A, E, B und G. Es ist zu prüfen, dass die Multiplikation der Blöcke tatsächlich den grauen Bereich der Ergebnismatrix ergibt.

In Abbildung 3.5 wird der grau hervorgehoben Bereich der Berechnungsvorschrift zufolge berechnet. Zunächst werden die Blöcke A mit E und B mit G zusammen sortiert. Das Ergebnis der beiden Multiplikationen ist in der folgende Zeile zu finden. Das Ergebnis der Addition der Zwischenergebnisse in der untersten Zeile. Die unterste Zeile enthält den grauen Block.

A	E	B	G
1	2	17	18
5	6	21	22
3	4	7	8
25	26	29	30
$A * E$		$B * G$	
59	62	191	198
211	222	407	422
$A * E + B * G$			
250	260		
618	644		

Abbildung 3.5: Matrix Multiplikation Blöcke A, E, B und G

Mit dem Wissen über die blockweise Aufteilung ist es nun möglich per SIMD zu parallelisieren oder Strip-Mining auszuführen, wenn keine parallelen Rechenknoten verfügbar sind. In dieser Arbeit werden Instruktionen analog zur RVV bereitgestellt um das Strip-Mining für Matrizen zu ermöglichen und dabei blockweise vorzugehen.

Es ist noch zu klären, wie die einzelnen, inneren Blöcke miteinander multipliziert und addiert werden.

3.6 Outer-Produkt

Das Outer-Produkt wird verwendet um die Matrixmultiplikation von Blöcken auszuführen. Die Eigenschaft, die das Outer-Produkt nützlich macht ist, dass es die Matrixmultiplikation auf die Berechnung von Additionen und Multiplikationen in einer gleichförmigen Form reduziert. Damit lässt sich das Verfahren einfach implementieren.

Das Outer-Produkt Verfahren erhöht auch die Parallelität im Vergleich zur Berechnung einzelner Zellen der Ergebnismatrix über einzelne Skalarprodukte. Beim Outerprodukt wird ein gesamtes Feld pro Durchgang berechnet. Nach allen Durchgängen enthält das Feld dann die Ergebnismatrix der Matrixmultiplikation.

In der Übersicht [9] von Earl A. Killian wird das Outer-Product auf Seite 53 beschrieben.

"The outer product is a fully parallel computation of mn multiplications."

Das Outer-Produkt wird aus zwei Vektoren u und v berechnet.

$$u = \begin{bmatrix} u_1 \\ u_2 \\ \vdots \\ u_m \end{bmatrix}, v = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{bmatrix}$$

Das Symbol ist $\otimes$ und das Ergebnis ist eine $m \times n$ Matrix C . Die Rechenvorschrift ist:

$$C = u \otimes v = \begin{pmatrix} u_1 v_1 & u_1 v_2 & \cdots & u_1 v_n \\ u_2 v_1 & u_2 v_2 & \cdots & u_2 v_n \\ \vdots & \vdots & \ddots & \vdots \\ u_m v_1 & u_m v_2 & \cdots & u_m v_n \end{pmatrix} \quad (3.2)$$

Nachdem das Outer-Produkt definiert ist, lässt das Outer-Produkt aufaddieren, um die Matrixmultiplikation zu beschreiben.

"Using this formulation, the matrix product can be expressed as the sum of K outer products of the columns of A with the rows of B "

$$C = AB = A_1^* \otimes B_*^1 + A_2^* \otimes B_*^2 + \dots + A_M^* \otimes B_*^N = \sum_{p=1}^k A_p^* \otimes B_*^p \quad (3.3)$$

In der Formel (3.3) kann man erkennen, dass es nicht ausreicht die Multiplikationen aus Formel (3.2) auszuführen, vielmehr müssen die Ergebnisse auch aufaddiert werden. Es handelt sich um eine Multiply-Accumulate Operation.

Daher ist es notwendig im Design Speicherzellen vorzusehen, die Schrittweise durch Addition vervollständigt werden, bis das Endergebnis vorliegt. Dieser "Matrix-Kern" aus Speicherzellen ist in Abbildung 3.6 aus [9] dargestellt. Dieses Konstrukt wird auch Matrix Accumulator (MAC) genannt, da er Zwischenergebnisse aufsummiert, also kummuliert.

Figure 1 4×4 Outer Product MAC Array with Local Accumulators

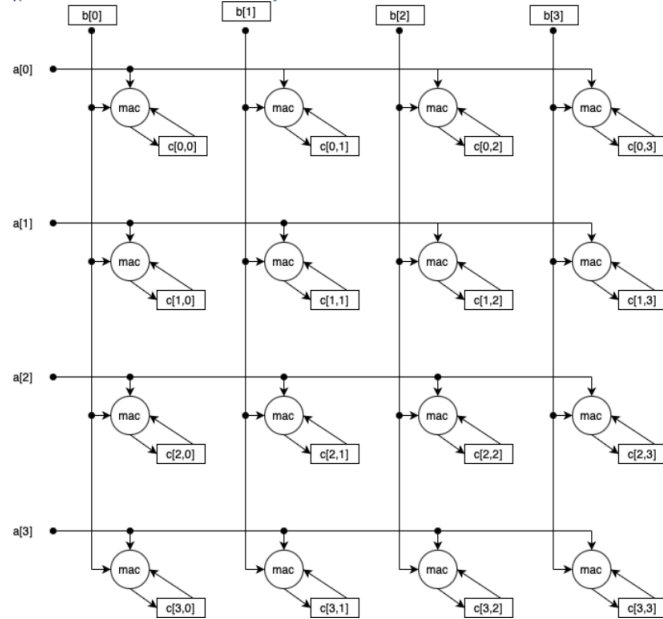


Abbildung 3.6: Matrix Accumulator (MAC)

Wenn zwei Vektoren außen an den MAC angelegt werden, dann berechnet er ein Zwischenergebnis für das Outer Product aus den beiden Vektoren (siehe Formel (3.2)) und akkumuliert das Zwischenergebnis zum Rest auf (siehe Formel (3.3)). Wenn alle Vektoren angelegt worden sind, enthält der MAC somit das komplette Ergebnis der Multiplikation.

Die Kombination aus dem blockweisen Ansatz und dem Outer Produkt mittels MAC ermöglicht es ein Design zu erstellen, dass dem angestrebten RISC-V AME Ansatz ermöglicht.

4 Durchführung und Methodik

Das Ziel ist beschrieben. Es geht um die Erweiterung eines RISC-V Kerns um eine hardwarebeschleunigte Matrixmultiplikation. Dabei soll die Arbeit möglichst nahe an einer der möglichen Erweiterungen liegen, die momentan tatsächlich von den RISC-V Gremien vorbereitet werden.

Es wurde besprochen, mit welcher Technik die Matrixmultiplikation ausgeführt werden soll und welche theretischen Grundlagen hinter den Überlegungen stehen.

In diesem Kapitel wird beschrieben, welches Vorgehen gewählt wird um die Arbeit durchzuführen.

4.1 Der NEORV32 Core und dessen Einsatz

Der NEORV32 Chip ist ein VHDL- und auch ein Verilog-Design von Stephan Nolting und anderen Programmierern¹, welches als Open-Source Projekt unter der BSD-3 Lizenz veröffentlicht ist [12]. Es gibt zum einen sehr gute Dokumentation zu dem Projekt in Form von textueller Dokumentation, dem sogenannten Datenblatt [13]. Zum anderen ist auch der Quellcode selbst sehr gut dokumentiert und einheitlich formatiert und damit gut lesbar.

Zunächst ist zu sagen, dass man den VHDL- oder Verilog-Quellcode auf unterschiedliche Arten nutzen kann. Dabei sind die unterschiedlichen Möglichkeiten interessant für unterschiedliche Phasen eines Projekts.

Im Zuge der Durchführung dieser Arbeit wird der Quellcode zunächst durch einen Simulator evaluiert. Als Simulator kommt GHDL in der Version 5.1.1 zum Einsatz. Die Evaluierung ermöglicht dann die Simulation des Designs für einen bestimmten Zeitschritt, beispielsweise für eine Millisekunde.

Die Simulation wird mit Eingangssignalen "stimuliert". Das Design verarbeitet dann die eingehenden Signale und der Simulator speichert die Signaländerungen in .vcd Dateien. Ein sogenannter Waveform-Viewer kann die Signale dann anzeigen. Durch visuelle Inspektion kann nun geprüft werden, ob das Design so auf die Eingabedaten reagiert, wie vom Designer erwartet.

Das NEORV32 Projekt bietet ein vorgefertigtes Skript, welches den Simulator GHDL ausführt. Also solches ist das NEORV32 Projekt für Anfänger sehr gut geeignet, da es diese Funktion bereits automatisch bereitstellt.

¹<https://github.com/stnolting/neorv32/graphs/contributors>

Wenn die Simulation die erwarteten Ergebnisse zeigt, dann kann ein Schritt weitergegangen werden. Das Design wird nun synthetisiert statt evaluiert. Die Synthese-Werkzeuge führen das sogenannte "Lowering" durch. "Lowering" im Sinne des Wortes soll andeuten, dass die Synthese-Werkzeuge eine Transformation von den abstrakten, also hohen Beschreibungen im Quellcode in Designs aus physischen Logikzellen durchführen. Von abstrakt zu physisch findet also ein "Lowering" statt. Sind die Logikzellen bestimmt, lokalisiert und miteinander verbunden, dann kann diese Aufteilung durch eine Bitstream-Datei auf einen FPGA übertragen werden.

Um die Synthese-Werkzeuge einzusetzen, gibt es ein Parallelprojekt zum NEORV32-Projekt namens, `neorv32-setups`. Dieses Projekts enthält ein TCL-Script, welches in der Entwicklungsumgebung Vivado in der Version 2025.1 ausgeführt werden kann und dann wiederum ein Vivado-Projekt erzeugt. Wenn dieses erzeugte Vivado-Projekt dann schließlich in Vivado 2025.1 geöffnet wird, dann kann der VHDL-Quellcode übersetzt werden.

Dabei führt Vivado seine gesamte Synthese-Toolchain durch und führt damit das Lowering konkret aus. Das Ergebnis ist die Bitstream-Datei. Vivado kann mit dem FPGA Board ARTY A7 100T sprechen und die Bitstream-Datei über eine USB-Verbindung auf den FPGA übertragen. Der FPGA wird sich dem Bitstream zufolge konfigurieren und ab dann läuft die NEORV32 CPU auf dem FPGA, als ob er auf einem Wafer produziert worden wäre.

Das Vorgehen in dieser Arbeit ist es zunächst die Änderungen in das NEORV32-Design einzubringen und zu simulieren, bis die Simulation laut Waveform-Diagramm erfolgreich ist. Danach erfolgt die Synthese mit Vivado.

Erfahrungsgemäß verhält sich das synthetisierte Design anders als das simulierte. Die Arbeit ist also nach der Simulation nicht getan. Die Umgestaltung des Designs nach der Simulation bis zur Funktion nach der Synthese wird nochmals als eigener Arbeitsschritt bei der Durchführung dieser Arbeit antizipiert.

4.2 Test-Software

Um zu Evaluieren, ob das Design wirklich funktioniert, muss die CPU mit einer Anwendungssoftware betrieben werden, die die neuen Instruktionen wirklich ausführt.

Die Anwendung muss in Form von Maschinencode, also kodierten Befehlen vorliegen. Eine Anweisung ist bei RISC-V eine 32-bit Zahl oder eine 16-bit Zahl, wenn komprimierte Anweisungen unterstützt werden. Die Software kann durch einen Assembler in Maschinencode übersetzt werden. Tatsächlich ist die Kodierung für RISC-V nicht kompliziert und dazu auch gut dokumentiert.

Trotzdem wurde der Einsatz eines Assemblers für das Vorgehen bei dieser

Arbeit nicht gewählt, da das Erstellen eines Assembler Programms ziemlich kompliziert ist. Es bestand die Furcht davor viel Zeit zu verlieren, wenn das Assemblerprogramm angepasst werden muss oder wenn mehr als eine Testsoftware erstellt werden soll. Dann ist für jede Anpassung wieder Assemblerentwicklung notwendig. Direkte Assembler-Entwicklung wird daher nicht verwendet.

Der erste Ansatz an ein lauffähiges Programm zu kommen, war es einen C-Compiler für eine Untermenge der Programmiersprache C zu erstellen. Der Vorteil der sich aus diesem Ansatz ergibt ist es zum einen volle Kontrolle darüber zu haben, welche RISC-V Anweisungen der Compiler ausgibt. Damit lässt sich ein RISC-V Programm erstellen, das tatsächlich auf der NEORV32 CPU ausgeführt werden kann, wenn man nur Anweisungen generiert, die der NEORV32 auch unterstützt.

Zum anderen lassen sich in den Compiler dann auch neue Anweisungen einpflegen, die im RISC-V Standard noch nicht definiert sind. Zwangsweise kennt ein GCC Compiler diese Anweisungen folglich auch nicht. Die Erweiterung des GCC Compiler ist um ein vielfaches komplizierter als die Erweiterung eines simplen, selbsterstellten Compilers. Die Unterstützung eigener Anweisungen wird in dieser Arbeit benötigt um die Matrix-Anweisungen verwenden zu können, die im Zuge dieser Arbeit hinzugefügt werden und daher von GCC nicht unterstützt werden.

Da die C Programmiersprache viele syntaktische Elemente und eine Vielzahl an Datentypen enthält, wurde der unterstützte Umfang der Sprache für den eigenen Compiler eingeschränkt. Es gibt nur einen einzigen Datentyp, einen 32 bit Integer. Damit entfällt jegliche Prüfung von Datentypen. Unterstützt werden Funktionsdefinitionen mit Parametern und Aufrufe dieser Funktionen. Dies hat zur Folge, dass der Compiler Stack-Frames erzeugen und auch wieder entfernen können muss. Zusätzlich gibt es for-Schleifen aber keine if-Anweisung, da für eine Matrixmultiplikation mit dem Outer-Produkt keine if-Anweisung benötigt wird. Zur Beschreibung der Matrizen unterstützt der Compiler Arrays vom Integer-Datentyp. Es gibt außerdem einen einfachen Präprozessor, der Define-Anweisungen ersetzen und include-Anweisungen ausführen kann.

Der Compiler erzeugt ein Listing aus Assembler-Befehlen, die dann von einem Assembler in Maschinen-Code übersetzt werden müssen. Es gibt keine Ausgabe von Objekt-Dateien und keine Linker-Infrastruktur und damit auch keine Bibliotheken. Alle Programme müssen komplett in Form von Quellcode vorliegen und können nicht aus vorübersetzten Bibliotheken stammen.

Es wurde ein beträchtlicher Zeitaufwand in diesen Compiler gesteckt und die Teilaufgabe wurde tatsächlich gelöst. Eine Testsoftware zur Matrixmultiplikation lässt sich mit dem Compiler aus der Untermenge von C in RISC-V Assembler übersetzen. Die Erstellung mehrerer Testanwendungen und deren schnelle Anpassung ist effizient möglich.

Zu einem späteren Zeitpunkt wurde zusätzlich auch das Verfahren von Stephan Nolting angewandt. Stephan Nolting hat das Problem von neuartigen Anweisungen auch unter Verwendung des bestehenden GCC Compilers ge-

löst, indem er Inline-Assembly verwendet um damit Maschinen-Code direkt aus dem Compiler ausgeben zu können. Damit hat er die Möglichkeit geschaffen auch Maschinencode für Anweisungen mit dem GCC zu erstellen, obwohl der GCC diese Anweisungen nicht kennt.

Der Nachteil ist, dass die Verwendung etwas kryptisch ist und viel Erfahrung mit dem GCC voraussetzt. Ein weiterer Nachteil des Inline-Assembly Ansatzes ist, dass der Compiler jetzt nicht mehr autonom arbeiten kann, sondern von einem Menschen gesteuert wird. Damit wird der Compiler davon abgehalten den Quellcode zu optimieren. Für Testfälle ist das Vorgehen zielführend. Für einen industriellen Einsatz muss der Compiler jedoch auf längere Sicht um die Instruktionen erweitert werden um die Optimierung wieder zu ermöglichen.

Das gewählte Vorgehen ist es, die Matrix-Erweiterung mit einer kleinen Testanwendung zu testen, die mit der Strategie von Stephan Nolting und dem GCC übersetzt wird.

4.3 Evaluierung der Performance

Die Hardware-beschleunigte Matrix-Berechnung ergibt nur Sinn, wenn sich ein Vorteil gegenüber einer Matrixmultiplikation ohne die Beschleunigung ergibt. Um zu bestimmen, ob sich die Matrix-Erweiterung positiv auswirkt, soll ein Vergleich zwischen zwei Anwendungen durchgeführt werden.

Die Anwendung A berechnet eine Matrixmultiplikation unter Verwendung von drei For-Schleifen. Die Anwendung B führt dieselbe Matrixmultiplikation unter Verwendung der neu hinzugefügten Anweisungen aus. Für beide Anwendungen wird gezählt wie viele CPU-Zyklen die Ausführung benötigt. Es wird selbstverständlich erwartet, dass die Anwendung B die Aufgabe mit weniger CPU-Zyklen erledigt.

Die Voraussetzungen für dieses Vorgehen ist, dass sowohl Anwendung A als auch Anwendung B in den Instruktions-Speicher des NEORV32 passen.

4.4 Schrittweises Vorgehen

Das Vorgehen besteht also aus folgenden Schritten:

1. Analyse der theoretischen Grundlagen zur Matrixmultiplikation. Es soll eine Möglichkeit gefunden werden, die die Berechnung auf einem FPGA ermöglicht und mit VHDL beschrieben werden kann.
2. Analyse der NEORV32 CPU. Es sollen die Komponenten der CPU identifiziert werden mit dem Ziel die Stellen zu finden, die geändert bzw. erweitert werden müssen.

3. Die Änderungen werden eingebracht.
4. Das geänderte Design wird simuliert bis die gewünschten Signale vorliegen.
5. Das geänderte Design wird synthetisiert und auf dem FPGA ausgeführt. Das Design wird angepasst, bis es auf dem FPGA funktioniert.
6. Ein Compiler (selbst erstellt oder der GCC Compiler) wird verwendet um Testprogramme zu übersetzen.
7. Die Testprogramme werden ausgeführt. Die Zyklenzahl wird gemessen und verglichen.

5 Analyse der NEORV32 CPU

Das Ziel dieses Kapitels ist die Analyse des bestehenden NEORV32 Cores. Die wichtigsten Elemente werden beschrieben. Durch die Analyse wird Klarheit darüber geschaffen, wie die Wirkungsweise der NEORV32 CPU ist und wie das System erweitert werden muss um die Matrixmultiplikation zu ergänzen.

5.1 Architektur des NEORV32 Core

Der NEORV32 besitzt den in Abbildung 5.1 dargestellten Aufbau. Diese Abbildung stammt aus dem NEORV32 Datenblatt [13].

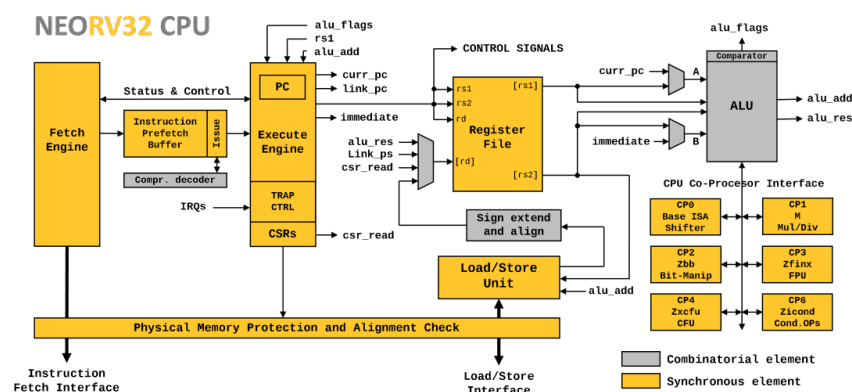


Abbildung 5.1: NEORV32 Architektur

Er ist im Groben aus einem Frontend und einem Backend aufgebaut. Das Frontend ermittelt die nächste Anweisung und das Backend führt diese Anweisung dann aus. Beide Teile arbeiten in gewisser Weise unabhängig voneinander. Damit kann das Frontend bereits neue Instruktionen puffern während das Backend die aktuelle Anweisung abarbeitet. Im Falle von Sprüngen kann sich der Puffer dann mit den neuen Anweisungen am Sprungziel füllen, während das Backend parallel arbeitet.

5.2 Die Fetch-Engine

Die Fetch-Engine steht am Anfang der Abarbeitung einer Anweisung. Sie ist für das Laden der Anweisung aus dem Speicher zuständig und lädt von der

Adresse, die momentan im Program-Counter-Register steht. Die Fetch Engine arbeitet unabhängig vom Rest der CPU. Sie implementiert intern eine State-Machine, die Anweisungen puffern kann und sie kann intern Anweisungen laden ohne, dass die Anweisungen akut vom Rest des Systems gebraucht wird.

Aus diesem Grund gibt es ein weiteres Signal, welches in Abbildung 5.2 als "debug_valid_i" sichtbar gemacht wurde und angibt, ob die geladene Instruktion gültig ist und verwendet werden sollte oder nicht.

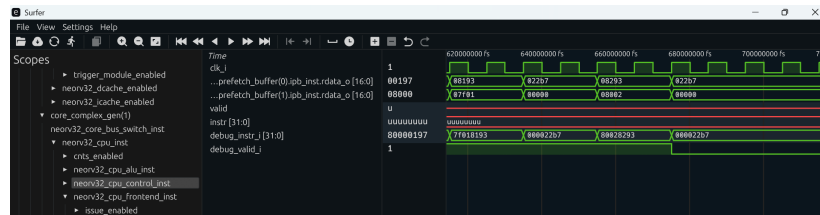


Abbildung 5.2: NEORV32 Gültige Instruktion

Dieses Wissen ist wichtig, wenn man das Verhalten der CPU anhand einer WaveForm-Ansicht diagnostizieren möchte. Dabei geht man oft von einem Assembler-Programm aus, das von der CPU ausgeführt wird. Wenn die Fetch-Engine kreuz und quer Instruktionen lädt, muss man wissen, ob die momentan anliegende Instruktion gültig ist oder nicht.

Es ist des Weiteren zu bemerken, dass die Fetch Engine nur über einen Cache verfügt wenn dies konfiguriert wird. In der Standardkonfiguration ist kein Cache für Instruktionen konfiguriert, wie in der Abbildung 5.1 dargestellt. Die Fetch Engine greift direkt auf den Speicher zu.

5.3 CPU-Control und Execution-Engine

Die Entität "CPU-Control" enthält die Kontrolllogik der CPU. Die CPU-Control enthält den Program Counter der auf die nächste Anweisung zeigt, die "Control and Status Register" (CSR) und Funktionen für Interrupts und Exceptions.

Bei einigen CPU-Designs für FPGAs findet man eine Kontrolllogik vor. Die Kontrolllogik besteht oft aus kombinatorischer Logik und gibt damit einfach direkt Signale aus, die ohne Zustand also ohne Speicherelemente direkt aus den Eingabesignalen berechnet werden. Ein Beispiel ist das Design aus [7]. Allgemein wird die Kontrolllogik Teile der CPU an- und abschalten. Damit konfiguriert sich eine CPU für jede Anwendung passend neu um diese Anweisung ausführen zu können.

Sehr häufig kommen Pipelines zum Einsatz. Eine Instruktion durchläuft nacheinander die Stages der Pipeline. Die freien Stages können für andere Instruktionen verwendet werden. Damit erhöht sich der Durchsatz des Systems und die Auslastung der vorhandenen Hardware erhöht sich.

Die NEORV32 CPU ist an dieser Stelle eine Ausnahme. Sie besitzt anstelle einer Pipeline eine State-Machine, die Execution-Engine genannt wird, um die CPU zu kontrollieren. Statt einer Fetch-Stage gibt es das separate Frontend, welches Instruktionen lädt.

Die Execution-Engine ist im Inneren eines VHDL-Prozesses definiert. Die allermeisten Prozesse enthalten das Reset-Signal und das Clock-Signal in ihrer Sensitivitätsliste. Der Prozess für die Execution-Engine ist anders. Hier befindet sich kein Clock-Signal in der Sensitivitätsliste sondern alle Signale, die die Execution-Engine beeinflussen, müssen explizit aufgelistet werden! Dies wird wichtig, wenn die Execution-Engine später erweitert wird. Beispielsweise muss die Execution-Engine nachdem sie einen Request nach außen gibt, auch wieder auf dessen Response warten. Das Response-Signal muss in die Sensitivitätsliste aufgenommen werden.

Der Start-Zustand ist der EX_RESTART state. Aus dem Reset geht die State-Machine in EX_BRANCHED und erst dann in den EX_DISPATCH Leerlaufzustand über. Der Umweg über EX_BRANCHED erlaubt es auf ein Instruction Fetch zu warten.

Im Leerlaufzustand EX_DISPATCH, wartet die Execution Engine bis ein Trap oder eine Exception eintritt oder bis das Frontend eine gültige Instruktion meldet, die dann abgearbeitet werden muss. Für normale Anweisungen wird der Zustand EX_EXECUTE betreten.

In EX_EXECUTE wird der nächste Zustand abhängig von der Art der Anweisung gewählt. Das heißt EX_EXECUTE ist ein Switch-State der in mehrere Äste der State-Machine verzweigen kann. Es gibt Zweige für ALU-Operationen, Sprünge und Speicherzugriffe (Load und Store).

Für länger laufende Befehle muss die Execution-Engine auf den Abschluss der Operation warten. Daher sind lange laufende Operationen jeweils durch ein Paar aus Request und Response Zuständen modelliert. Der Request Zustand wird direkt in Richtung Response-Zustand verlassen. Der Response-Zustand wird erst in Richtung EX_DISPATCH verlassen, wenn das eingehende Response-Signal aus der Sensitivitätsliste aktiviert worden ist. Im Response-Zustand werden die erzeugte Daten betrachtet und Signale zurückgesetzt.

5.4 Die Load-Store-Engine

Die State-Machine der Execution-Engine produziert eine Vielzahl an Signalen, die andere Entitäten des VHDL-Designs aktivieren und mit Eingabedaten versorgen. Ein wichtiger Teil des Designs ist die sogenannte Load-Store Unit. Die Load-Store Unit hat zur Aufgabe Bus-Requests zu erstellen und Bus-Responses zu verarbeiten.

Die Bus-Requests werden auf dem Bus innerhalb der NEORV32 CPU ausgeführt. Ein Teilnehmer am Bus ist z.B. der Datenspeicher. Mit Datenspeicher

ist der Teil des Speichers gemeint, der die Werte der Variablen speichert, also konkret keinen Quellcode sondern eben Daten enthält. Der Datenspeicher wird über Load-Anweisungen gelesen und produziert damit Daten für die Register oder er wird durch Store-Anweisungen geschrieben. Die Aufgabe der Load-Store-Unit besteht darin die Signale der Execution-Engine in passende Bus-Requests für Load- oder Store Anweisungen zu übersetzen.

Dabei ist folgende Besonderheit zu beachten. Die Bus-Requests, die von der Load-Store-Unit erzeugt werden sind in der Datei `neorv32_top.vhd` mit dem Data-Cache verbunden. Damit unterstützt die NEORV32 CPU eine Cache-Hierarchie für Daten. Anstatt direkt auf den Speicher zuzugreifen, greift die Load-Store-Unit immer zunächst in den Cache. Wenn ein Cache-Hit stattfindet, ergibt sich ein Geschwindigkeitszuwachs.

6 Erweiterung der NEORV32-CPU

In diesem Kapitel werden die Erweiterungen an den Entitäten des vorgegebenen Designs beschrieben, die gemacht wurden um die Matrix-Multiplikation zu ermöglichen.

6.1 Die Matrix-Register

Die Matrixregister bestehen in der jetzigen Version aus 16 32-Bit Elementen und sind nicht für andere Datentypen konfigurierbar und nicht für größere Längen miteinander kombinierbar, wie es bei der RVV der Fall ist. Die Register sind wie folgt definiert.

```
type ram_type is array (0 to MLEN-1)
  of std_ulogic_vector(XLEN-1 downto 0);
subtype matrix_ram_addr is integer range 0 to MLEN-1;

signal matrix_ram_0 : ram_type;
signal matrix_ram_1 : ram_type;
signal matrix_ram_2 : ram_type;
```

6.2 Die Instruktionen

Um die Matrixmultiplikation durchzuführen sollen neue Instruktionen hinzugefügt werden, die es noch nicht bereits in anderen Extensions für RISC-V gibt. Die Instruktionen sollen das Strip-Mining für Matrizen ermöglichen und sich dabei an der RVV-Extension orientieren.

Bei der Diskussion des Strip-Mining bei der RVV-Extension wurde die Instruktion `set(i)vli` besprochen, mit der die Vektor-Engine um Datentypen und gewünschte Datenlängen konfiguriert wurde. Des weiteren wurden die Instruktionen `vle32.v`, `vadd.vv` und `vse32.v` im Rahmen des Beispiel-Codes verwendet.

`vle32.v` lädt Daten aus dem Daten-Speicher in die Vector-Register (Load-Operation), `vadd.vv` ist die SIMD Instruktion, die zwei Vektoren elementweise parallel miteinander addiert hat und `vse32.v` speichert die Daten schließlich in den Daten-Speicher zurück (Store-Operation)

Für die Matrix-Extension werden die Instruktion `mle32`, `mmul32` und `mse32`

hinzugefügt. Dabei ist `mle32` die Load-Operation, `mmul32` die SIMD - Instruktion und `mse32` die Store-Operation. Ein Äquivalent zur `set(i)vli` ist nicht vorgesehen, da die Matrix-Engine zum jetzigen Stand keine Konfigurationsmöglichkeiten anbietet.

Der NEORV32 besitzt eine Menge an Beispielen, die sich im Software-Unterordner befinden. Jedes Beispiel kann von Funktionen Gebrauch machen, die über die `neorv32.h` Header-Datei eingebunden werden können. Teil dieser Funktionen sind sogenannte Intrinsics. Ein Intrinsic ist eine Funktion, die in der Programmiersprache C geschrieben ist und die Programmiersprache C damit um Funktionalität erweitert, die thematisch zu einem anderen Themengebiet gehört.

Ein Beispiel dafür sind die Intrinsics the X-Window Systems [5]. Hier wird eine einfach zu verwendende Schnittstelle zur Softwareentwicklung mit dem X-Window System für die Programmiersprache erstellt.

Die Intrinsics des NEORV32 sind eine Erweiterung der Programmiersprache C um RISC-V Instruktionen aus C-Quellcode heraus auszugeben. Ein Beispiel für die Verwendung ist das folgende Testprogramm.

```

1  #include <neorv32.h>
2
3  uint32_t matrix_a[16]
4    = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16 };
5
6  uint32_t matrix_b[16]
7    = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16 };
8
9  int main() {
10
11     uint32_t *matrix_a_addr = &matrix_a[0];
12     uint32_t *matrix_b_addr = &matrix_b[0];
13
14     uint32_t matrix_unit_a = 0;
15     uint32_t matrix_unit_b = 0;
16
17     matrix_unit_a = CUSTOM_INSTR_I_TYPE(0x000, matrix_a_addr, 0b010
18                                         , 0b1011011);
19
20     matrix_unit_b = CUSTOM_INSTR_I_TYPE(0x001, matrix_b_addr, 0b010
21                                         , 0b1011011);
22
23     uint32_t matrix_mul = CUSTOM_INSTR_I_TYPE(0x000, matrix_a_addr,
24                                                0b100, 0b1011011);
25
26     matrix_unit_a = CUSTOM_INSTR_I_TYPE(0x000, matrix_a_addr, 0b001
27                                         , 0b1011011);
28     matrix_unit_b = CUSTOM_INSTR_I_TYPE(0x001, matrix_b_addr, 0b001
29                                         , 0b1011011);
30
31     for (int i = 0; i < 16; i++) {
32         neorv32_uart0_printf("After Matrix A %d\r\n", matrix_a_addr[i
33                               ]);
34     }
35
36     for (int i = 0; i < 16; i++) {
37         neorv32_uart0_printf("After Matrix B %d\r\n", matrix_b_addr[i
38                               ]);
39     }
40
41     return 0;

```


33 }

In der Zeile 17 ff wird als Intrinsic "CUSTOM_INSTR_I_TYPE" für I-Type RISC-V Instruktionen verwendet. Aus jeder Zeile mit einem "CUSTOM_INSTR_I_TYPE" Aufruf wird eine RISC-V Assembler Instruktion generiert.

Die Verwendung ist schwer zu verstehen, daher wird jetzt beschrieben, wie die Parameter zum Intrinsic zu interpretieren sind. Der letzte Parameter ist der Op-Code. Für die Matrix-Extension wurde in dieser Arbeit der Op-Code 0b1011011 gewählt. Innerhalb dieses Opcodes werden dann durch den vorletzten Parameter die drei Anweisungen mle32, mmul32 und mse32 kodiert.

Es ist noch abschließend zu erklären, weshalb Intrinsics verwendet werden müssen. Der grundsätzliche Ablauf bei der Verwendung der Programmiersprache C ist, dass ein Compiler den Quellcode der höheren Programmiersprache in Eigenregie in Assembler-Code übersetzt. Dabei wählt er die Anweisungen selbst aus. Optimierende Compiler führen diese Aufgabe in vielen Fällen besser aus, als es ein Mensch könnte.

Im Fall dieser Arbeit ist dieses Vorgehen nicht möglich. Der C-Compiler kennt die neu definierten Instruktionen mle32, mmul32 und mse32 schlicht nicht und wird sie daher nur ausgeben, wenn man den Compiler explizit dazu anweist. Diese explizite Anweisung passiert durch die Intrinsics.

Intern sind die Intrinsics durch Inline Assembly implementiert. Das bedeutet, dass dem C-Compiler exakt vorgegeben wird, welchen inline assembly Code er auszugeben hat. Im Beispiel-Code werden die Instruktionen mle32, mmul32 und mse32 per Opcode generiert.

Im Zuge dieser Arbeit ist ein einfacher Compiler erstellt worden, der auch dazu verwendet werden kann, die neuen Instruktionen auszugeben. Dieser Compiler wurde verwendet, bis der Weg über die Intrinsics des NEORV32 entdeckt wurde. Die Intrinsics erlauben es State-of-the-Art Compiler wie den GCC zu verwenden. Dies ist sehr viel komfortabler als einen eigenen Compiler zu pflegen.

6.3 Erweiterung der Execution-Engine

Bis zu diesem Punkt liegt ein Beispiel-Programm mit den neuen Matrix-Instruktionen vor, welches in die NEORV32 CPU geladen und ausgeführt werden kann.

In diesem Abschnitt wird die Execution-Engine erweitert, so dass sie die neuen Instruktion auch ausführt, wenn sie von dem Frontend präsentiert werden.

Eine Übersicht aller Komponenten ist in Abbildung 6.1 zu sehen.

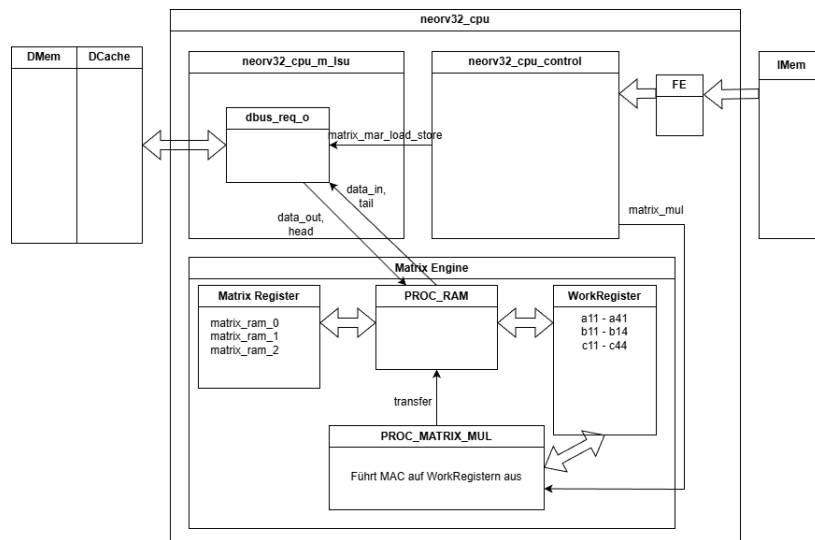


Abbildung 6.1: VHDL Entities

Die Instruktionen werden über das Frontend (FE) aus dem Anweisungsspeicher (IMem) geladen und an die Kontrolllogik weitergegeben. Die Daten der Matrix-Elemente kommen aus dem Datenspeicher (DMEM) und durchlaufen den Daten-Cache (DCache).

Die Komponente `neorv32_cpu_control` gibt die Kontrolsignale `matrix_mar_load_store` und `matrix_mul` aus, wenn die Instruktionen `mle32`, `mse32` bzw. `mmul32` ausgeführt werden müssen.

Im Falle von `mle32` (Load) werden die Signale `data_out` und `head` verwendet um die 16 Matricelemente vom Bus-Request an die richtige Stelle in die Matrix-Register zu übertragen. `PROC_RAM` übernimmt diese Koordinationsaufgabe.

Wird eine `mmul32` Anweisung ausgeführt und damit das `matrix_mul` signal von der Control-Logik gesetzt, dann beginnt der Prozess `PROC_MATRIX_MUL` mit der Multiplikation. Die Matrix-Register werden in die Arbeitsregister übertragen. Nach der Bearbeitung wird das Ergebnis aus den Arbeitsregistern wieder zurück in die Matrixregister übertragen.

Analog zu `mle32` werden die Signale `data_in` und `tail` für `mse32` (Store) Anweisung verwendet um Daten aus den Matrixregistern zurück in den Datenspeicher und den Cache zu schreiben.

Im Folgenden werden die Änderungen im Detail beschrieben.

Als erstes wird der neue Matrix-Opcode aus der Erkennung illegaler Opcodes ausgeschlossen.

```
-- MATRIX EXTENSION
when opcode_matrix_c =>
    illegal_cmd <= '0';
```

Der Execution-Engine wird ein neues Signal "matrix_operation_wait_i" in die Sensitivitätsliste eingefügt. Dadurch kann sie das Ende der Matrixberechnung erkennen und in den nächsten Zustand übergehen.

```
execute_engine_fsm_comb: process(
    ...
    matrix_operation_wait_i
    ...
)
```

Der ALU wird beigebracht, dass sie den Operand B als immediate verwenden muss, sobald sie den Matrix-Extension Opcode sieht. Der Grund ist, dass die Load- und Store-Matrix-Anweisungen die Register-Id als immediate Wert verwenden um zu ermitteln, welches Matrix-Register Sie lesen und schreiben sollen. Damit muss die ALU diesen Immediate-Wert weitergeben, sonst ist er verloren und kann nicht mehr verwendet werden.

```
-- ALU operand B: is immediate? --
case opcode is
    -- add matrix opcode here because the mle32, mse32 instructions
    -- contain an immediate
    -- that needs to be added to the register encoded in the
    -- instruction
    when opcode_matrix_c | opcode_alui_c | opcode_lui_c ...
        ctrl_nxt.alu_opb_mux <= '1';
    when others =>
        ctrl_nxt.alu_opb_mux <= '0';
end case;
```

Im EX_DISPATCH Zustand wird die Matrixmultiplikation gestoppt.

```
matrix_mul_o <= '0';
```

Der EX_EXECUTE Zustand wird um die Behandlung der Matrix-Instruktionen erweitert.

```
-- Matrix Extension --
when opcode_matrix_c =>

case funct3_v is

    -- load from memory into matrix unit --
    when funct3_mle32_c =>
        -- go to the specific states for the matrix extension.
        -- The states are inserted in order to load all the elements
        -- of a matrix in a loop
        exe_engine_nxt.state <= EX_MATRIX_MEM_REQ;

    when funct3_mse32_c =>
        exe_engine_nxt.state <= EX_MATRIX_MEM_REQ;

    when funct3_mmul32_c =>
        -- go to the state that processes matrix operations
        exe_engine_nxt.state <= EX_MATRIX_OPERATION_REQ;
```

```

when others => -- undefined
end case;

```

Für die drei Instruktionen mle32, mse32 und mmul32 werden neue Zustände in die Execution Engine eingefügt, so dass diese Zustände aus EX_EXECUTE angesprungen werden können.

Für die Matrix Load- und Store-Instruktionen wird der Zustand EX_MATRIX_MEM_REQ hinzugefügt.

```

-- trigger a loop of memory requests for the matrix
when EX_MATRIX_MEM_REQ =>

    -- tell the matrix unit to come online and process the request
    ctrl_nxt.lsu_m_en    <= '1';

    if (or_reduce_f(trap_ctrl.exc_buf(exc_ialign_c downto
        exc_iaccess_c)) = '0') then

        -- memory request if no instruction exception

        -- tell the load store unit to process a request
        ctrl_nxt.lsu_req    <= '1';

        -- forward the immediate from the instruction
        matrix_immediate_o <= immediate;

        -- rs1 (source register 1) carries the address to load from
        / store to
        matrix_mar_load_store_o <= rf_rs1_i;

        exe_engine_nxt.state <= EX_MATRIX_MEM_RSP;

    else
        -- On Error
        exe_engine_nxt.state <= EX_DISPATCH;
    end if;

```

Im Zustand EX_MATRIX_MEM_RSP wird auf die Rückmeldung gewartet. Wenn die Rückmeldung anhand des Signals "lsu_wait_i" erfolgt ist, dann wird der geladene Wert über das write-back signal (wb) in die Register-File (RF) geschrieben. Dann wird die Matrix-Engine deaktiviert.

```

when EX_MATRIX_MEM_RSP => -- wait for memory response

    if (lsu_wait_i = '0') then

        -- write-back to RF if read operation (won't happen in case
        of exception)
        ctrl_nxt.rf_wb_en    <= (not ctrl.lsu_rw) or ctrl.lsu_rvs or
            ctrl.lsu_rmw;

        -- tell the matrix unit to stay offline
        ctrl_nxt.lsu_m_en    <= '0';

        exe_engine_nxt.state <= EX_DISPATCH;
    end if;

```

```
end if;
```

Für Matrixoperationen gibt es die beiden neuen Zustände EX_MATRIX_OPERATION_REQ und EX_MATRIX_OPERATION_RSP. Mit Matrix Operationen ist die Matrixmultiplikation gemeint. Da die Matrixmultiplikation nicht in einem einzigen Schritt ausgeführt ist, muss die Execution-Engine auf die Matrix-Engine warten. Daher ergibt sich die Notwendigkeit für einen Request- und einen Response-State.

```
when EX_MATRIX_OPERATION_REQ => -- matrix add operation

  if (or_reduce_f(trap_ctrl.exc_buf(exc_ialign_c downto
    exc_iaccess_c)) = '0') then

    matrix_mul_o <= '0';

    case funct3_v is

      when funct3_mmul32_c =>
        matrix_mul_o <= '1';

      when others => -- undefined

    end case;

    matrix_immediate_o <= immediate;

    -- next state
    exe_engine_nxt.state <= EX_MATRIX_OPERATION_RSP;

  else
    exe_engine_nxt.state <= EX_DISPATCH;
  end if;

when EX_MATRIX_OPERATION_RSP =>

  -- next state
  if (matrix_operation_wait_i = '0') then

    matrix_mul_o <= '0';

    exe_engine_nxt.state <= EX_DISPATCH;

  end if;
```

Diese beiden Zustände sind sehr simpel. Im Request-State wird das Signal matrix_mul_o aktiviert und dann in den Response-State übergegangen. Im Response-State wird auf das Signal matrix_operation_wait_i gewartet und dann wird die Matrixmultiplikation deaktiviert.

6.4 Erweiterung der Load-Store-Unit

Der nächste Schritt ist die Erweiterung der Load-Store-Unit. Die Load-Store-Unit ist innerhalb der CPU-Entity definiert.

Die Aufgabe besteht darin, Daten aus dem Datenspeicher in die neuen Matrix-Register zu laden. Hier wäre es möglich die Matrix-Engine als eigenen Bus-Teilnehmer an den Bus zu binden. Die Matrix-Engine hätte dann die Möglichkeit Daten aus dem Datenspeicher über den Bus zu laden.

Der Nachteil an diesem Ansatz ist, dass die Matrix-Engine und die Load-Store-Unit nicht mehr synchronisiert sind. Die Matrix-Engine wird Daten direkt aus dem RAM laden ohne auf den Cache zuzugreifen. Die Load-Store Unit hingegen verwendet den Cache weiterhin. Da die Load-Store-Unit verantwortlich für den Rest der CPU ist, greift die CPU dann insgesamt auf veraltete Daten zu.

Um die Synchronisierung wieder herzustellen, müsste die Load-Store-Unit ihren Cache nach Ausführung einer Matrix- Load-Store-Operation invalidieren, da dieser veraltete Daten enthalten könnte. Daraufhin muss sich der Cache erst wieder aufwärmen und die Performance der CPU leidet folglich.

Aus diesem Grund wurde das Laden der Matrix-Engine direkt in die LSU integriert. Wenn eine Matrixoperation eine Matrix fertig berechnet hat und diese Daten dann wieder in den Datenspeicher übertragen werden, dann durchlaufen die Daten den Cache der LSU und wärmen den Cache damit für den Rest der CPU mit aktuellen Daten auf. Das bedeutet, dass der Rest der ausgeführten Anwendung dann direkt auf die gecachte Matrix zugreifen kann anstelle die Matrix erst wieder aus dem Datenspeicher laden zu müssen.

Damit werden Daten über die Load-Store-Unit geladen, damit die Daten im Cache landen. Die Load-Store-Unit erzeugt Bus-Anfragen wenn Sie Daten laden und speichern soll. Da ein Matrix-Register aus 16 Elementen besteht, werden beide Prozesse, für Load und für Store erweitert um in einer Schleife 16 Elemente laden zu können.

Der Load-Prozess wird für Matrix-Load-Anweisungen um eine State-Machine erweitert. Der State ML_IDLE deaktiviert die Load-Operation. Der Zustand ML_INIT setzt den Zähler auf 0 und geht in den ML_PROCESS Status über. Im ML_PROCESS Status erfolgen 16 Load Operationen um das Matrix-Register zu füllen. Alle 16-Operation durchlaufen den Cache was positiv ist. Der ML_INCREMENT wird abwechselnd mit ML_PROCESS ausgeführt und zählt den Zähler nach oben.

Sind die 16 Operationen ausgeführt, geht die State-Machine zurück nach ML_IDLE.

Ein etwa gleichartiges Vorgehen wird auch für die Matrix-Store-Operation gewählt, welche 16 Elemente zurückschreiben muss.

Der neue PROC_RAM Prozess produziert Steuersignale für den RAM-Speicher der Matrix-Engine. Abhängig von den Zählern, die in den Operationen Load und Store verändert werden, produziert der PROC_RAM Prozess Signale, die die Load-Store-Unit verlassen und in der CPU mit dem RAM für die Matrix-Engine verbunden sind. Damit wird der RAM der Matrix-Engine gesteuert und gibt entweder Daten für ein Store zurück oder speichert Da-

ten für eine Load-Anweisung. Die eigentlichen Daten kommen aus dem Daten-Speicher der CPU und durchlaufen den Cache der LSU.

6.5 Matrix-Engine

In die Datei "neorv32_cpu.vhd" wird die Matrix-Engine eingebaut.

Die Engine besteht zunächst aus dem PROC_RAM Prozess.

```

PROC_RAM : process(clk_i)
    variable l : line;
begin
    if (rstn_i = '0') then
        ...
    elsif rising_edge(clk_i) then
        if (transfer = '1') then
            -- store the c temporaries into the second matrix engine
            register
            matrix_ram_0(0) <= std_ulogic_vector(c11(31 downto 0));
            matrix_ram_0(1) <= std_ulogic_vector(c12(31 downto 0));
            ...
            matrix_ram_0(15) <= std_ulogic_vector(c44(31 downto 0));

            -- store the c temporaries into the second matrix engine
            register
            matrix_ram_1(0) <= std_ulogic_vector(c11(31 downto 0));
            matrix_ram_1(1) <= std_ulogic_vector(c12(31 downto 0));
            ...
            matrix_ram_1(15) <= std_ulogic_vector(c44(31 downto 0));
        end if;
        if (load = '1') then
            -- data_out is the output of the LoadStoreUnit (LSU) and it
            -- contains the RAM/cache value
            -- which is loaded into the MatrixEngine
            if (to_integer(unsigned(matrix_immediate)) = 0) then
                matrix_ram_0(head) <= data_out;
            else
                matrix_ram_1(head) <= data_out;
            end if;
        end if;
        if (store = '1') then
            if (to_integer(unsigned(matrix_immediate)) = 0) then
                data_in <= matrix_ram_0(tail);
            else
                data_in <= matrix_ram_1(tail);
            end if;
        end if;
        if (matrix_add = '1') then
            for i in 0 to 15 loop
                matrix_ram_0(i) <= std_ulogic_vector(unsigned(matrix_ram_0(i)
                    )) + unsigned(matrix_immediate));
            end loop;
        end if;
    end if;
end process PROC_RAM;

```

Dieser Prozess hat die Aufgabe die Matrixregister zu speichern und zu laden

und die Rechenergebnisse der Matrixmultiplikation in das Zielmatrixregister zu übertragen (transfer). Es wäre auch möglich weitere Operationen wie ein elementweises Addieren einer Konstante in diesem Prozess zu implementieren.

Beim Speichern und Laden der Matrixregister werden die Signale head und tail verwendet, die von der LSU ausgegeben werden. Diese Signale beschreiben das Element, dass geschrieben werden soll.

Wenn die LSU Daten über den Bus aus dem RAM bzw. dem Cache erhält, dann wird gleichzeitig dazu das entsprechende Element im Register geschrieben oder gelesen. Die Synchronisation der LSU mit dem PROC_RAM Prozess geschieht über die Signale head und tail.

Der zweite Teil der Matrix-Engine besteht aus der Berechnung des Outer-Product. Dazu sind bei 16 Elementen pro Matrixregister und damit mit 4x4 Matrizen vier Schritte notwendig. In den vier Schritten werden Zeilen aus der Matrix A und Spalten aus der Matrix B miteinander verrechnet. Mit vier Zeilen und vier Spalten müssen also vier Berechnungen pro Schritt ausgeführt werden.

Um die Berechnung auszuführen wird eine State-Machine verwendet. Die State-Machine durchläuft die Zustände MM_RESET, MM_INIT_1, und MM_PROCESS_1, bis MM_INIT_4, MM_PROCESS_4 und endet mit MM_STORE, MM_DONE.

Der MM_RESET Zustand setzt die Matrixregister auf 0, wenn das matrix_mul Signal auf 1 gesetzt wird. Solange matrix_mul auf 0 steht, verbleibt der Wert der letzten durchgeführten Matrixmultiplikation in den Matrix Register stehen. Der Grund ist, dass man die Werte in den Register halten muss um sie in den RAM zurückschreiben zu können und dies einige CPU-Zyklen dauert. Das matrix_mul Signal wird von der Kontrolllogik der CPU gesetzt, wenn ein mmul32 Befehl ausgeführt wird und steht ansonsten auf 0.

Wenn das matrix_mul Signal gesetzt wird, werden die Matrixregister mit 0 überschrieben und die State Machine geht in den nächsten Zustand über. Es folgen vier Schritte, die jeweils aus einem INIT und einem PROCESS Schritt bestehen. Alle vier Schritte sind äquivalent, es wird daher nur ein Schritt beschrieben.

Im INIT Schritt wird eine Zeile aus der A-Matrix und eine Spalte aus der B-Matrix in extra Signale geladen:

```
when MM_INIT_1 =>
  a11 <= unsigned(matrix_ram_0(0));
  a21 <= unsigned(matrix_ram_0(4));
  a31 <= unsigned(matrix_ram_0(8));
  a41 <= unsigned(matrix_ram_0(12));

  b11 <= unsigned(matrix_ram_1(0));
  b12 <= unsigned(matrix_ram_1(1));
  b13 <= unsigned(matrix_ram_1(2));
  b14 <= unsigned(matrix_ram_1(3));
```



```
mat_mul_engine_state <= MM_PROCESS_1;
```

Im folgenden MM_PROCESS Schritt erfolgt die sogenannte Multiply - Accumulate - Operation oder kurz MAC. Mit Multiply-Accumulate ist gemeint, dass eine Multiplikation als Kernoperation ausgeführt wird aber das Ergebnis dieser Multiplikation nur ein Teilinkrement einer größeren Operation darstellt. In diesem Fall ist die größere Operation die Matrixmultiplikation, die sich aus vier addierten (kumulierten) Outer-Produkten zusammengesetzt. Die Outer-Produkte werden in den Akkumulator-Speichern c11 bis c44 aufgenommen.

```
when MM_PROCESS_1 =>

    c11 <= c11 + (a11 * b11);
    c12 <= c12 + (a11 * b12);
    c13 <= c13 + (a11 * b13);
    c14 <= c14 + (a11 * b14);

    c21 <= c21 + (a21 * b11);
    c22 <= c22 + (a21 * b12);
    c23 <= c23 + (a21 * b13);
    c24 <= c24 + (a21 * b14);

    c31 <= c31 + (a31 * b11);
    c32 <= c32 + (a31 * b12);
    c33 <= c33 + (a31 * b13);
    c34 <= c34 + (a31 * b14);

    c41 <= c41 + (a41 * b11);
    c42 <= c42 + (a41 * b12);
    c43 <= c43 + (a41 * b13);
    c44 <= c44 + (a41 * b14);

    -- next state
    mat_mul_engine_state <= MM_INIT_2;
```

Nach vier MM_INIT und vier MM_PROCESS Zuständen ist die Matrixmultiplikation durchgeführt und wird schließlich im MM_STORE Schritt in das Zielregister übertragen. Dazu wird das Signal "transfer" auf 1 gesetzt.

```
when MM_STORE =>
    -- enable transfer in the RAM part
    transfer <= '1';
```

Das "transfer" signal wird im PROC_RAM Prozess gelesen und führt dort dazu, dass die MAC - Speicherzellen c11 bis c44 in das Ziel-Matrixregister kopiert werden. Dieser Sachverhalt wurde bereits weiter oben bei der Beschreibung des PROC_RAM Prozesses beschrieben.

Es soll an dieser Stelle kurz bewertet werden, ob die Berechnung der vier Outer-Produkte auf diese Weise sinnvoll ist oder nicht. Als erstes ist diese Lösung funktional und diese Qualität ist sicherlich sehr wichtig. Ein VHDL Design zu erstellen, dass auch wirklich nach der Synthese auf einem FPGA funktioniert ist speziell für Anfänger immer ein großer Erfolg.

Als zweites ist der Quellcode vom Standpunkt der VHDL-Synthese aus zu betrachten. Aus Sicht einer höheren Programmiersprache ist die Implementierung nicht sehr sinnvoll, da man beispielsweise durch das Strassen-Verfahren einen Geschwindigkeitsvorteil erzielen kann. VHDL ist aber keine höhere Programmiersprache sondern eine Hardwarebeschreibungssprache. Dinge können in Hardware tatsächlich parallel ablaufen, wenn Elemente mehrfach synthetisiert werden.

Die 16 Multiplikationen pro MM_PROCESS Zustand werden hoffentlich von der Toolchain auf Hardware-Blöcke innerhalb des FPGA abgebildet und laufen damit effizient und wirklich parallel ab. Wenn der FPGA mehrere Hardware-Blöcke für die Multiplikation hat, dann können diese Multiplikationen tatsächlich parallel ausgeführt werden.

Zusätzlich wäre es denkbar die vier MM_INIT und MM_PROCESS Schritte nicht sequentiell an eine State-Machine zu binden sondern sich über die parallele Ausführung der vier Schritte Gedanken zu machen. Dazu muss die Aufgabe gelöst sein eine Parallele Akkumulation der Teilergebnisse in die Speicher c11 bis c44 zu bewerkstelligen. Ein vierfaches, gleichzeitiges Schreiben in die Zellen ist ohne Synchronisation vermutlich nicht möglich. Aus diesem Grund wurde im ersten Schritt das sequentielle Vorgehen gewählt. Hier kann weiter optimiert werden.

Damit sind die wichtigsten Erweiterung des NEORV32 beschrieben.

7 Auswertung und Evaluierung

In diesem Kapitel wird untersucht, ob der gewählte Ansatz zur Integration von Matrixoperationen in die NEORV32 CPU zielführend war. Zunächst wird die Funktionalität der Erweiterung geprüft. Das Prüfverfahren wird erläutert.

Danach wird untersucht, ob das erweiterte System eine Matrixmultiplikation schneller ausführen kann, als das unmodifizierte System. Um diesen Vergleich anzustellen wird die mathematische Vorschrift zur Matrixmultiplikation mit drei for-Schleifen auf zwei Matrizen ausgeführt. Die gleichen beiden Matrizen werden mit den hinzugefügten Anweisungen zur Matrixmultiplikation verrechnet.

Das Messverfahren wird beschrieben. Die Laufzeiten werden gemessen, ausgewertet und verglichen. Das Ergebnis dieses Vergleichs wird eingeordnet und diskutiert und es wird ein abschließendes Fazit gezogen.

7.1 Testdaten

Um eine Vergleichbarkeit der Verfahren und eine Überprüfung der Korrektheit zu erzielen, werden Testdaten mit erwartetem Ergebnis vorgegeben. In diesem Fall handelt es sich um zwei Matrizen A und B von denen das Ergebnis der Multiplikation bekannt ist und durch Matrix C dokumentiert wird.

Es wird eine 4x4 Matrixmultiplikation mit den beiden Matrizen A und B durchgeführt, für die das Ergebnis bekannt ist.

$$C = A * B \quad (7.1)$$

$$\begin{bmatrix} 90 & 100 & 110 & 120 \\ 202 & 228 & 254 & 280 \\ 314 & 356 & 398 & 440 \\ 426 & 484 & 542 & 600 \end{bmatrix} = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix} \times \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix}$$

$$\begin{bmatrix} 5A & 64 & 6E & 78 \\ CA & E4 & FE & 118 \\ 13A & 164 & 18E & 1B8 \\ 1AA & 1E4 & 21E & 258 \end{bmatrix} = \begin{bmatrix} 90 & 100 & 110 & 120 \\ 202 & 228 & 254 & 280 \\ 314 & 356 & 398 & 440 \\ 426 & 484 & 542 & 600 \end{bmatrix}$$

Zur einfacheren Vergleichbarkeit der Matrix C mit der Waveform der Simulation wurde die Matrix C noch in hexadezimaler Schreibweise angegeben, da die Waveform Zahlen hexadezimal darstellt.

7.2 Messung und Vergleichbarkeit

Um Anweisungsfolgen miteinander zu vergleichen, muss zunächst eine Skala gewählt werden. Danach müssen die Anweisungsfolgen auf die Skala abgebildet werden. Die Abbildung muss dem Quellcode eine Zahl auf der Skala zuordnen. Als letztes muss für die Skala dann noch erwähnt werden, welcher Wert besser ist, also wie die Qualität von der Skala abzulesen ist.

Als Skala wird die Zahl der Clock-Ticks gewählt, die die CPU zur Ausführung der Anweisungsfolge benötigt. Die Abbildung erfolgt über ein Feature, dass die NEORV32 CPU liefert. Es handelt sich hier um Register, die die Clock-Ticks zählen. Die Qualität auf der Skala ist dadurch bestimmt, dass weniger Clock-Zyklen besser sind. Die CPU löst eine Aufgabe dann schneller.

Es wird nun das Zählen der Clock-Ticks beschrieben. In RISC-V Terminologie gibt es sogenannte CSR. CSR steht für Control- und Statusregister. Diese Register werden nicht bei der Abarbeitung von Anweisungen direkt verwendet. Sie können indirekt Einfluss auf die Arbeitsweise der CPU haben, da sie dessen Zustand enthalten. Über bestimmte Befehle lassen sich die CSR ändern und damit lässt sich der Zustand der CPU konfigurieren.

Ein Paar von CSR, dass bei der Zählung der Clock-Zyklen verwendet wird sind die Register CSR_MCYCLE und CSR_MCYCLEH. Das erste Register (CSR_MCYCLE) speichert das Low-Word und das zweite Register (CSR_MCYCLEH) speichert das High-Word des Clock-Cycle-Zählers.

Es werden wirkliche Clock-Cycles gezählt und keine Anweisungen. Das ist ein Vorteil, da damit eine wirkliche Messung erzielt werden kann. Wenn Anweisungen gezählt werden, ist nicht klar wieviel Zeit die CPU zur Bearbeitung der Anweisungen benötigt.

Ein Clock-Cycle beschreibt einen Ausschlag der externen Clock-Source. Die externe Clock-Source wird an das VHDL-Design als Input-Signal herangeführt. Dieses Signal wird in sehr vielen Prozessen des Designs verwendet um sequentielle, getaktete Logik voranzutreiben. Die NEORV32 CPU kann beispielsweise mit einer Taktrate von 100 Mhz betrieben werden. Damit ist die externe Clock-Source dann auf 100 MHz eingestellt. CSR_MCYCLE und CSR_MCYCLEH werden dann mit einer Taktrate von 100 Mhz inkrementiert.

Hier ist ein Beispiel für die Verwendung der Abbildung auf die Cycle-Skala:

```
1 // start timing
2 neorv32_cpu_csr_write(CSR_MCYCLE, 0);
3
```

```

4 for (i=0; i<(DATA_NUM/2); i++) {
5     v[0] = input_data[i*2+0];
6     v[1] = input_data[i*2+1];
7     xtea_sw_encipher(XTEA_ROUNDS, v, key);
8     cypher_data_sw[i*2+0] = v[0];
9     cypher_data_sw[i*2+1] = v[1];
10 }
11
12 // stop timing
13 uint32_t time_enc_sw = neorv32_cpu_csr_read(CSR_MCYCLE);
14
15 // -----
16 // Print timing results
17 // -----
18 neorv32_uart0_printf("\nExecution timing:\n");
19 neorv32_uart0_printf("ENC SW=%u cycles\n", time_enc_sw);

```

Im Beispiel wird nur das Low-Word und damit nur CSR_MCYCLE verwendet. Die Erwartungshaltung ist offensichtlich, dass die gemessene Befehlsfolge berechnet ist, bevor der Zähler in das High-Word überläuft.

Da das CSR_MCYCLE CSR-Register einen beliebigen Wert enthält, muss es zunächst in Zeile 2 auf 0 zurückgesetzt werden. Dann kann sein Wert nach Abarbeitung der Befehlsfolge in Zeile 13 ausgelesen und schließlich über eine serielle Schnittstelle ausgegeben werden.

Dieses Verfahren wird im Folgenden zur Bewertung der unterschiedlichen Matrixmultiplikationen verwendet. Es wird dabei inklusive der Übetragung der Daten von und zum CPU-RAM gemessen, da der Zugriff auf den CPU-RAM bei der Hardwarebeschleunigung berücksichtigt werden kann und damit ein Geschwindigkeitsvorteil erzielt werden kann.

Das Vorgehen ist das Folgende:

1. In Vivado wird das Design synthetisiert. Dabei wird der Bootloader als erstes Anwendungsprogramm direkt in das Design synthetisiert. Damit ist der Bootloader-Maschinencode direkt im internen Instruktion-Speicher der CPU enthalten und wird damit direkt beim Systemstart ausgeführt.
2. Der Bitstream wird über den in Vivado integrierten Hardware-Manager auf das FPGA-Board übertragen und dort ausgeführt. Der Bootloader ist jetzt über eine UART-Schnittstelle erreichbar.
3. Für jedes Testprogramm, wird dieses Testprogramm mit dem GCC Compiler in der Version riscv-none-elf-gcc.exe (xPack GNU RISC-V Embedded GCC x86_64) 14.2.0 übersetzt. Für die Übersetzung der Beispiele und Testprogramme sind im NEORV32 Projekt Skripte und auch das sogenannte "image_gen" Werkzeug bereitgestellt. Mit "image_gen" wird aus den Ausgaben des GCC eine Datei erstellt, die vom Bootloader verstanden wird.
4. Die Ausgabe des "image_gen" Werkzeugs wird dem Bootloader über die UART-Schnittstelle zugeführt. Dazu wurde ein eigener Uploader er-

stellt. Die Firmware könnte aber auch über jeden beliebigen Terminal-Emulator übertragen werden.

5. Der Bootloader speichert das übertragene Programm im NEORV32 CPU und führt es aus, sobald der Uploader den Befehl zur Ausführung gibt.
6. Über die UART-Schnittstelle wird die gemessene Anzahl an Clock-Zyklen gelesen und notiert.
7. Die Ausgaben aller Testprogramme werden verglichen und bewertet.

7.3 Prüfung mit Matrix-Erweiterung

Für die Prüfung mit Hardwarebeschleunigung, enthält das ausgeführte Anwendungsprogram die `mle32`, `mmul32` und `mse32` Anweisungen als Intrinsics.

Die Matrix A und die Matrix B werden aus dem CPU-RAM in die Matrix-Engine geladen. Danach erfolgt die Multiplikation durch Aufruf der `mmul32` Anweisung, danach erfolgt das Speichern der Matrix A und Matrix B zurück in den CPU RAM. Matrix A enthält in der momentanen Implementierung das Ergebnis der Multiplikation.

```
1 #include <neorv32.h>
2
3 uint32_t matrix_a[16]
4 = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16 };
5
6 uint32_t matrix_b[16]
7 = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16 };
8
9 int main() {
10
11     uint32_t *matrix_a_addr = &matrix_a[0];
12     uint32_t *matrix_b_addr = &matrix_b[0];
13
14     uint32_t matrix_unit_a = 0;
15     uint32_t matrix_unit_b = 0;
16
17     neorv32_cpu_csr_write(CSR_MCYCLE, 0);
18
19     matrix_unit_a = CUSTOM_INSTR_I_TYPE(0x000, matrix_a_addr, 0b010
20                                         , 0b1011011);
21     matrix_unit_b = CUSTOM_INSTR_I_TYPE(0x001, matrix_b_addr, 0b010
22                                         , 0b1011011);
23
24     uint32_t matrix_mul = CUSTOM_INSTR_I_TYPE(0x000, matrix_a_addr,
25                                               0b100, 0b1011011);
26
27     matrix_unit_a = CUSTOM_INSTR_I_TYPE(0x000, matrix_a_addr, 0b001
28                                         , 0b1011011);
29     matrix_unit_b = CUSTOM_INSTR_I_TYPE(0x001, matrix_b_addr, 0b001
30                                         , 0b1011011);
31 }
```

```

27  uint32_t time_enc_sw = neorv32_cpu_csr_read(CSR_MCYCLE);
28
29  neorv32_uart0_printf("\nExecution timing:\n");
30  neorv32_uart0_printf("ENC SW = %u cycles\n", time_enc_sw);
31
32  return 0;
33 }

```

Zunächst wird über den WaveForm Viewer visuell geprüft, ob die korrekten Steuersignale ausgegeben werden. Dazu wird Abbildung 7.1 betrachtet.

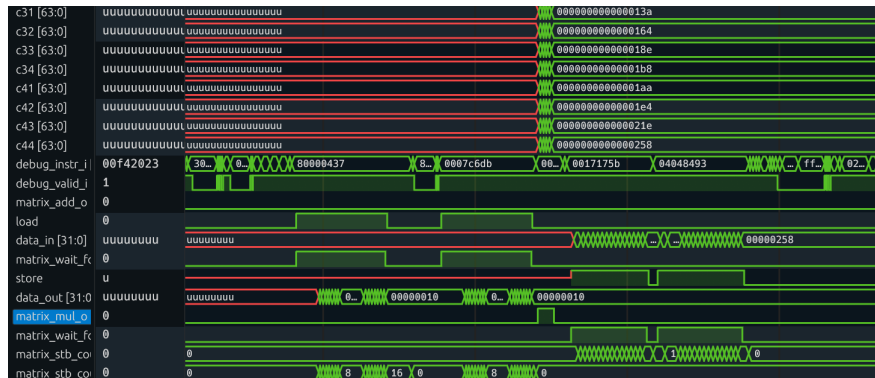


Abbildung 7.1: Steuersignale der Matrixmultiplikation

Die schnellste Art Informationen aus der Ansicht zu ziehen ist es, sich auf die Signale "load", "matrix_mul_o" und "store" zu konzentrieren. Diese Signale entsprechen der Abarbeitung der Instruktionen mle32, mmul32 und mse32 in genau dieser Reihenfolge. In der Abbildung 7.1 kann man sehen, dass die Signale in genau der gewünschten Reihenfolge und auch Häufigkeit auf den Wert eins springen. Es wird zwei mal geladen und gespeichert, da die Testanwendung die Matrizen A und B lädt und speichert. Die Operation mmul32, angedeutet durch das Signal "matrix_mul_o", wird nur einmalig ausgeführt, wie erwartet.

Danach wird geprüft, ob die Matrixmultiplikation in der Simulation mit den richtigen Daten abgearbeitet wird und ob die richtigen Werte in der Matrix C abgelegt werden.

In Abbildung 7.2 ist zu sehen, dass sich über die Zeit hinweg neue Daten in den Registern a und b befinden. Es ist auch zu sehen, dass die Matrix-Engine die Werte aus den Registern a und b über das Outer-Produkt multipliziert und auf die Accumulator-Register c aufaddiert. Schließlich ist nach den vier Schritten zu sehen, dass das korrekte Ergebnis in den Accumulator Registern c steht und dort auch für einige Zeit stehen bleibt, so dass die Daten weiterverarbeitet werden können und nicht direkt gelöscht werden.

Abschließend lässt sich sagen, dass die hardwarebeschleunigte Matrixmultiplikation mit dem erstellten Design wie geplant funktioniert.

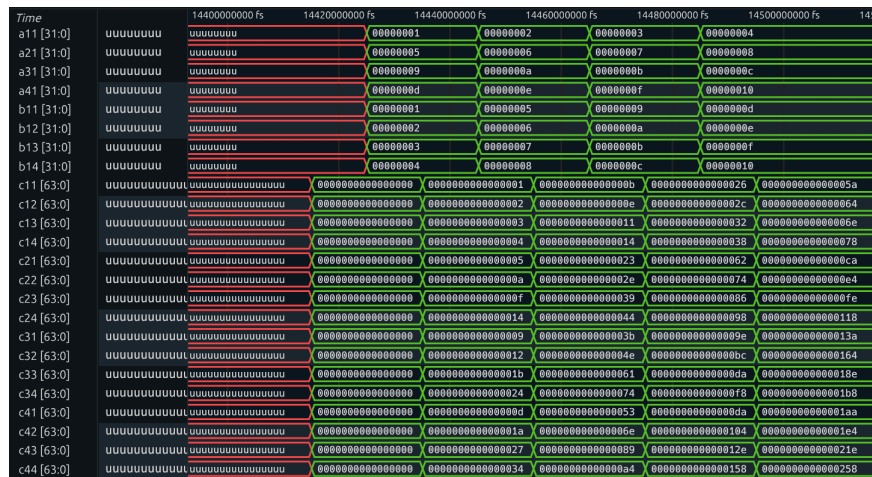


Abbildung 7.2: Waveforms der Matrixmultiplikation

7.4 Prüfung ohne Matrix-Erweiterung

Das Testprogramm für die Matrixmultiplikation ohne Hardwarebeschleunigung ist:

```

1 #include <stdio.h>
2
3 #define DIMENSION 4
4 #define ELEMENTS DIMENSION*DIMENSION
5
6 int standardMatrixMult(int* matrixA, int* matrixB, int* matrixC,
7     int rows, int columns) {
8
9     int counter = 0;
10
11     // over row of matrix B
12     for (int i = 0; i < rows; i++) {
13
14         // over column of matrix A
15         for (int j = 0; j < columns; j++) {
16
17             // fuse row and column together into a single cell of
18             // matrix C
19             for (int k = 0; k < columns; k++) {
20
21                 // int aldx = i * columns + k;
22                 int temp_aldx = i * columns;
23                 int aldx = temp_aldx + k;
24
25                 // int bldx = k * columns + j;
26                 int temp_bldx = k * columns;
27                 int bldx = temp_bldx + j;
28
29                 // int cldx = i * rows + j;
30                 int temp_cldx = i * rows;
31                 int cldx = temp_cldx + j;
32
33                 int aVal = matrixA[aldx];
34                 int bVal = matrixB[bldx];
35
36                 // int cVal = matrixC[cldx];
37                 int temp_cVal = cldx;
38                 int cVal = temp_cVal + aVal * bVal;
39
40                 matrixC[cldx] = cVal;
41
42                 counter++;
43             }
44         }
45     }
46
47     return counter;
48 }
```



```

34         int mult_temp = aVal * bVal;
35
36         int temp_2 = matrixC[cldx];
37         int temp_3 = temp_2 + mult_temp;
38         matrixC[cldx] = temp_3;
39
40     }
41
42 }
43
44 }
45
46 return 0;
47 }
48
49 int main()
50 {
51     neorv32_cpu_csr_write(CSR_MCYCLE, 0);
52
53     int matrixA[ELEMENTS] = {
54         1, 2, 3, 4,
55         5, 6, 7, 8,
56         9, 10, 11, 12,
57         13, 14, 15, 16
58     };
59     int matrixC[ELEMENTS] = {
60         0, 0, 0, 0,
61         0, 0, 0, 0,
62         0, 0, 0, 0,
63         0, 0, 0, 0
64     };
65     int matrixB[ELEMENTS] = {
66         1, 2, 3, 4,
67         5, 6, 7, 8,
68         9, 10, 11, 12,
69         13, 14, 15, 16
70     };
71
72     int a22 = standardMatrixMult(matrixA, matrixB, matrixC, 4, 4)
73         ;
74
75     uint32_t time_enc_sw = neorv32_cpu_csr_read(CSR_MCYCLE);
76
77     // -----
78     // Print timing results
79     // -----
80     neorv32_uart0_printf("\nExecution timing:\n");
81     neorv32_uart0_printf("ENC SW = %u cycles\n", time_enc_sw);
82
83     int resultPrettyPrintC = prettyPrintFormatMatrix(matrixC,
84         DIMENSION);
85
86     exit();
87     return 0;
88 }

```

Der C-Quellcode ist sehr außergewöhnlich gestaltet, einzig und alleine weil der selbsterstellte C-Compiler diesen Quellcode in dieser Form auch übersetzen können soll, falls weitere Test durchgeführt werden sollten.

7.5 Auswertung

Für die hardwarebeschleunigte Variante erfolgt folgende Ausgabe:

```
1 "\nBooting from 0x00000000...\r"
2 "\n\r"
3 "\nMatrix Multiplication Sample \r\r"
4 "\n\r"
5 "\nExecution timing:\r"
6 "\nENC SW=185 cycles\r"
```

Es werden 185 Zyklen notiert.

Für die nicht hardwarebeschleunigte Variante erfolgt folgende Ausgabe:

```
1 "\nBooting from 0x00000000...\r"
2 "\n\r"
3 "\nMatrix Multiplication Sample \r\r"
4 "\n\r"
5 "\nExecution timing:\r"
6 "\nENC SW=10385 cycles\r"
```

Es werden 10385 Zyklen notiert.

Laut gemessenen Werten und rein mathematisch betrachtet wird die Matrixmultiplikation zweier 4x4, vorzeichenloser Integer-Matrizen unter Ausführung der Matrix-Extension um einen Faktor von etwa 56 beschleunigt.

Die Messung vergleicht im Grunde zwei Testanwendungen. Die Testanwendung mit den Anweisungen aus der Matrix-Extension lässt sich nicht mehr stark optimieren. Das Vergleichsprogramm mit der herkömmlichen Multiplikation ließe sich durch eine bessere Implementierung sicherlich stark vereinfachen. Damit ergibt sich dann ein anderes Verhältnis zwischen den Zyklen, die die Testprogramme verbrauchen. Es ist jedoch nicht abzusehen, dass sich das Verhältnis zu gunsten einer herkömmlichen Implementierung ohne Hardwarebeschleunigung wendet, auch wenn der Algorithmus stark optimiert wird.

Es lässt sich abschließend konstatieren, dass die Hardware-Beschleunigung einen positiven Einfluss auf die Laufzeit hat. Dies ist hauptsächlich auf die effiziente Übertragung der Elemente zwischen CPU-RAM und dem Prozess-RAM der Matrix-Engine zurückzuführen. Die 16 Datenelemente werden als Stream zwischen dem Speicher und der Matrix-Engine übertragen und dies geschieht ohne dass die Execution-Engine 16 einzelne Load- und Store Befehle ausführen muss. Der zusätzliche Aufwand wird damit gegenüber einer naiven Implementierung um ein Vielfaches reduziert. Der Geschwindigkeitszuwachs ist hauptsächlich auf diesen Effekt zurückzuführen.

8 Zusammenfassung und Ausblick

Am Ende der Arbeit wird das Erreichte nochmals zusammengefasst um von diesem Standpunkt aus einen Blick in die Zukunft zu wagen.

Rückblickend lässt sich sagen, welche Entscheidungen zielführend und welche Entscheidungen hinderlich waren. Dies wird in der Zusammenfassung diskutiert.

Für die Zukunft werden im Ausblick Ideen angeführt, die während oder nach der Arbeit an der Lösung entsanden sind. Diese Gedanken sollen als Wegweiser für eine eventuelle Verbesserung oder Weiterführung der Arbeit dienen.

8.1 Zusammenfassung

Im Rahmen dieser Arbeit wurde die existierende NEORV32 CPU um Komponenten zur Matrix-Berechnung erweitert. Die Komponenten unterliegen starken Einschränkungen. Unter anderem können zum jetzigen Zeitpunkt lediglich 4x4 Matrizen mit vorzeichenlosen 32-Bit Zahlen berechnet werden. Zum termingerechten Abschluss dieser Arbeit war es notwendig solche Einschränkungen vorzunehmen. Für die Zukunft wäre es durch zusätzlichen Arbeitseinsatz möglich das Design um weitere Datentypen und Konvertierungen zu erweitern.

Die durchgeführten Änderungen orientieren sich an der Befehlsmenge der bereits existierenden RISC-V Vektor-Extension und greifen damit auch das Konzept des Strip-Mining auf. Strip-Mining erlaubt die Erstellung eines Hardware-Software-Co-Designs, welches in der Lage ist Matrixmultiplikationen größerer Matrizen durch Multiplikationen kleinerer Dimensionen zusammenzusetzen.

Um die Softwareentwicklung für Anwendungen und Testprogramme zu ermöglichen, wurden exzessive Anstrengungen unternommen, die etwa den gleichen Zeitaufwand wie die VHDL Entwicklung zur Folge hatten. Insbesondere wurde es durch einen Compiler ermöglicht die neuen Anweisungen auch unter Verwendung einer höheren Programmiersprache wie C in RISC-V Maschinencode zu übersetzen. Im Nachhinein wäre der schnellstmögliche Einsatz der NEORV32 Intrinsics ein sehr großer Vorteil gewesen. Leider wurde dieser Weg erst spät gelernt.

Es wurde ein Vergleich zweier Testprogramme auf dem NEORV32 Chip durchgeführt. Die Messungen zeigen einen Geschwindigkeitszuwachs um

einen relevanten Faktor. Das Experiment ist geglückt und könnte in Zukunft durch die Erweiterung auf ein vollständiges Typsystem und weiter verbesserte Speicherzugriffe noch verbessert werden.

8.2 Ausblick

Am Ende dieser Arbeit drängt sich eine grundlegende Frage auf. In welchen Szenario ist es sinnvoll Hardwarebeschleuniger in ein bestehendes RISC-V Design einzupflegen und ab wann sollte sich ein Unternehmen für die Erstellung eines komplett neuen Designs entscheiden. Diese Frage ist eng verknüpft mit der Frage ob der in dieser Arbeit gewählte Ansatz richtig oder falsch ist.

Im Zuge dieser Arbeit wurde ein bestehendes Design erweitert. Der Zugriffsweg auf den Speicher über die Load-Store-Unit und den am Bus angeschlossenen Speicher wurde wiederverwendet. Der Vorteil ist es sicherlich Kosten und Zeit sparen zu können, da existierende Komponenten wiederverwendet werden. Ein Nachteil ist eventuell, dass die wiederverwendeten Komponenten auf Anforderungen hin erstellt und optimiert worden sind, die in der Vergangenheit maßgebend waren aber eventuell neue Erweiterungen nicht effizient unterstützen. Für diese Arbeit ist dieser Umstand nicht zu einem Problem geworden.

Die Alternative wäre es, ein grundlegend neues Design zu beginnen welches dann die neuen Anforderungen besser abdeckt. Ein Beispiel für einen neuen Anforderung ist es beispielsweise mehr als einen Speicher-Zugriff parallel ausführen zu können. Die Hardware für die Outer-Produkt Berechnung könnte dann mehrmals synthetisiert werden. Jede Instanz der Outer-Produkt Berechnung könnte möglicherweise über ihren eigenen Zugriff auf den Speicher verfügen. Wenn der Speicher echt-parallel Daten über mehrere Ports gleichzeitig liefern kann, wird eine Matrix-Berechnung parallel statt sequenziell ausgeführt.

Ein Nachteil eines solch spezialisierten Designs ist es, dass es seine Allgemeinheit verliert. Ein RISC-V CPU soll eine allgemeine Rechenmaschine bleiben.

Es ist fallabhängig, welchen Weg ein Unternehmen wählen sollte. Ein finanzstarkes Unternehmen kann sich für die Erstellung von Anwendungsspezifischen Chips entscheiden, wie man es im Falle von Amazon und dem Trainium Chip sehen kann ¹. Unternehmen wie NVIDIA erstellen Beschleunigerkarten, die für High-Performance Computing im Sinne von großer Parallelität eingesetzt werden können und damit ein größeres Feld abdecken. CPU Designs von Intel und AMD sind um Instruktionserweiterungen wie AVX2, AMX und FMA für die Vektor- und Matrixberechnung erweitert worden wobei sie immer noch allgemeines Computing erlauben. In Zukunft wird RISC-V diesen Weg mit einer Matrix-Extension auch gehen und hat mit der RVV bereits begonnen.

¹<https://aws.amazon.com/de/ai/machine-learning/trainium/>

Wahrscheinlich ist es aus wirtschaftlicher Sicht notwendig zu wissen, was der Markt fordert und Hardware erstellen zu können die die Anforderung des Marktes erfüllen. Aus akademischer Sicht ist es wichtig die Zusammenarbeit mit Firmen aus der Industrie zu suchen um innovative Lösungen für die zukünftigen Entwicklungen finden zu können.

Als nächste Schritte zur Fortführung dieser Arbeit ist es denkbar die Matrix-Extension intelligenter und damit performanter zu gestalten. Der Vergleich mit anderen Designs wie z.B. [11] und [14] führt zu neuen Erkenntnissen und einen Zuwachs an Erfahrung.

Zusätzlich ist das System aus dieser Arbeit um weitere Datentypen zu ergänzen. Es gibt traditionelle Datentypen wie beispielsweise 8-, 16-, 32- und 64-bit ganzzahlige Datentypen mit und ohne Vorzeichen und auch 32- und 64-bit Gleitkommatypen. Dazu kommen neuartige Aufteilungen von Daten, wie sie für Large Language Models verwendet werden. Ein Beispiel ist hier NF4, was für normalized float mit 4-Bit steht. Sobald die Matrix-Extension mit unterschiedlichen Datentypen verwendet werden können soll ist über die Konvertierung zwischen den Datentypen nachzudenken. Es müssen ebenfalls dazu passende Instruktionen ergänzt werden. Die `set(i)vl` Instruktion aus der RVV-Extension benötigt dann ein Äquivalent in der Matrix-Extension um die Matrix-Extension für Datentypen konfigurierbar zu machen.

Es ist zu prüfen, welche Ergebnisse das Lowering hervorgebracht hat. Wurden wirklich Hardware-Bausteine innerhalb des FPGA verwendet oder sind Anteile unnötigerweise durch Logikzellen synthetisiert worden? Hier lässt sich in Kleinstarbeit noch eine Optimierung durchführen, falls Logikzellen anstatt von vorhandener Hardware verwendet werden.

Ein weiterer Schritt ist es sicherlich eine Umgebung für die Validierung des Systems bereitzustellen. Aufgrund zeitlicher Beschränkungen wurde das Design nur visuell mit einem Waveform-Viewer geprüft. Die Sprache SystemVerilog beispielsweise bietet weitreichendere Möglichkeiten zur Validierung. Für VHDL könnte die UVVM² oder cocotb³ eingesetzt werden.

²<https://www.uvvm.org/>

³<https://www.cocotb.org/>

Quellenverzeichnis

- [1] AKENINE-MÖLLER, Tomas ; HAINES, Eric ; HOFFMAN, Naty: Real-Time Rendering 3rd Edition. Natick, MA, USA : A. K. Peters, Ltd., 2008. – 1045 S. – ISBN 987-1-56881-424-7
- [2] ALON AMID, et. a. Krste Asanovic A. Krste Asanovic: RISC-V "V"Vector Extension. <https://github.com/riscvarchive/riscv-v-spec/blob/master/v-spec.adoc>. Version: 2024
- [3] ARSALAN, Qizal: NEORV32 Hardware Accelerator (Arty A7) – Modified Core with CFU/CFS. https://github.com/QizalA/neorv32_hardware_accelerator_arty_a7, 2025
- [4] ASANOVIC, Krste: RISC-V State of the Union. <https://riscv-europe.org/summit/2025/media/proceedings/2025-05-13-RISC-V-Summit-Europe-11h30-ASANOVIC%2086-slides.pdf>. Version: 2025
- [5] DAVID FLANAGAN, Adrian N.: X Toolkit Intrinsics Reference Manual: for X11 Release 4 and Release 5. The Associates, 1990
- [6] FALK, Sigurd: Ein übersichtliches Schema für die Matrizenmultiplikation. Berlin : Wiley-VCH, Zeitschrift für Angewandte Mathematik und Mechanik (ZAMM), 1951
- [7] HARRIS, Sarah L. ; HARRIS, David: Digital Design and RISC-V Computer Architecture Textbook. In: 2021 ACM/IEEE Workshop on Computer Architecture Education (WCAE) (2021), 1-5. <https://api.semanticscholar.org/CorpusID:246872104>
- [8] INTERNATIONAL, RISC-V: Integrated Matrix Extension. <https://riscv.atlassian.net/wiki/spaces/IMEX/pages/46071925/Charter>. Version: 2024
- [9] KILLIAN, Earl A.: VME Design Considerations. <https://riscv.atlassian.net/wiki/download/attachments/663781400/DesignConsiderationsForVME.pdf?api=v2>. Version: 2025
- [10] LARABEL, Michael: Qualcomm's Xqci RISC-V Extension Now Deemed Non-Experimental For LLVM 22. <https://www.phoronix.com/news/Qualcomm-Xqci-LLVM-Stable>. Version: 2025
- [11] MOREIRA, José E. ; BARTON, Kit ; BATTLE, Steven ; BERGNER, Peter ; BERT-RAN, Ramon ; BHAT, Puneeth ; CALDEIRA, Pedro ; EDELSON, David ; FOS-SUM, Gordon C. ; FREY, Brad ; IVANOVIC, Nemanja ; KERCHNER, Chip ;

- LIM, Vincent ; KAPOOR, Shakti ; FILHO, Tulio M. ; MUELLER, Silvia M. ; OLSSON, Brett ; SADASIVAM, Satish ; SALEIL, Baptiste ; SCHMIDT, Bill ; SRINIVASARAGHAVAN, Rajalakshmi ; SRIVATSAN, Shricharan ; THOMPTON, Brian W. ; WAGNER, Andreas ; WU, Nelson: A matrix math facility for Power ISA(TM) processors. In: CoRR abs/2104.03142 (2021). <https://arxiv.org/abs/2104.03142>
- [12] NOLTING, Stephan: neorv32. <https://github.com/stnolting/neorv32>. Version: 2026
- [13] NOLTING, Stephan: The NEORV32 RISC-V Processor - Data Sheet. <https://stnolting.github.io/neorv32/>. Version: 2026
- [14] OSS, Stream C.: riscv-matrix-project. <https://github.com/riscv-stc/riscv-matrix-project>, 2025
- [15] THOMAS, Giles: Basic matrix maths for neural networks: the theory. <https://www.gilesthomas.com/2025/02/basic-neural-network-matrix-maths-part-1>. Version: 2025
- [16] TILO ARENS, FRANK HETTLICH, CHRISTIAN KARPfINGER, ULRICH KOCKELKORN, KLAUS LICHTENEGGER, HELLMUTH STACHEL: Mathematik. SpringerSpektrum, Springer-Verlag, Berlin Heidelberg 2008, 2011, 2015, 2015
- [17] ZEE, Field G. ; GEIJN, Robert A.: BLIS: A Framework for Rapidly Instantiating BLAS Functionality. In: ACM Transactions on Mathematical Software 41 (2015), June, Nr. 3, 14:1–14:33. <https://doi.acm.org/10.1145/2764454>